

Phenix Overview and General Reference (Part 5 of 5)

16 documentation pages

Generated from local Phenix documentation

Contents

1. Using the Selection Editor
2. Guessing sequence from a map with `sequence_from_map`
3. Setting up tests that can run automatically in the Phenix GUI
4. Showing information about a map file with `show_map_info`
5. Homology modelling using `simple_homology_model`
6. Finding NCS in chains from a PDB file with `simple_ncs_from_pdb`
7. Sorting heteroatoms
8. `phenix.structural_domain_search`
9. Comparing related structures in the Phenix GUI
10. `structure_search`
11. Welcome to AI-powered Phenix and CCTBX programming!
12. Analyzing unmerged data in Phenix
13. Using AI to help generate doc strings
14. Helpful hints for using AI in coding for Phenix and CCTBX
15. `phenix.varref`: a tool for refinement of a model against multiple maps
16. Structure factor file manipulations with Xmanip

Using the Selection Editor

selection_editor.html

Contents

- Overview
- Basic Usage
- Graphical Selection
- Known Issues

Overview

The selection editor is a uniform interface for graphically selecting groups of atoms. The editor generates the syntax that Phenix uses for atom selections and is a more advanced interface than the "View/Pick" button next to certain fields in the GUI. This interface is used to define

- Secondary structure annotations (helices, beta sheets, etc.)
- NCS groups
- TLS groups
- Refinement strategy options (occupancies, rigid bodies, etc.)
- Custom geometry restraints (bonds, angles, dihedrals, etc.)

Moreover, selections made in this editor can be saved as a file ("Export" button) and reloaded at a later time ("Load" button). It is important to note that selection files saved from one Phenix program GUI (e.g. phenix.refine) generally cannot be used with another Phenix program GUI (e.g. phenix.real_space_refine) because the organization for the parameters can be slightly different among the programs.

The "Validate Syntax" button can be used to check if the selections are valid for the given model. This only checks for syntactical correctness. This does not check if the selections are physically sensible or if there are overlapping/duplicated selections.

The "Sequence" button opens a new window showing the sequence for the given model and any secondary structure annotations for helices and beta sheets.

The "Model" button opens a new window for graphically selecting atoms.

It is important to note that selections can only be made for options that have been enabled in the Phenix program GUI. That is, if the "Use NCS" box is not checked in the phenix.refine GUI, there will be no panel for selecting NCS groups in the selection editor. Changing options in the Phenix program GUI will dynamically update the available panels in the selection editor. If selections were made for an option and that option is later disabled, those selections will not be passed to the Phenix program when the selection editor is closed.

Closing the selection editor will update the Phenix program GUI with the selections. This is required before running the program.

Basic Usage

The selection editor can generate two basic types of selections

- Groups composed of one selection (e.g. helix or rigid body)

- Groups composed of multiple selections (e.g. beta sheet or NCS group)

For both types, the general procedure for adding a new group is to click on the "Add ..." button (e.g. "Add Sheet"). This generates a new row and the selection can be directly typed in the row by double-clicking the appropriate column field. For the second type, you can add additional selections to the group (e.g. "Add Strand") by entering selections in the new row. Some selections have additional columns and they can be changed by double-clicking the appropriate column field and typing in the value. To expand or collapse the multiple selections within an group, you can click on the "Expand All" or "Collapse All" buttons, respectively. Some times, it is more sensible to toggle through a series of choices for a parameter (e.g. "Sense" for beta strands) and double-clicking that field will do that instead.

Deleting groups behaves similarly to adding groups. Multiple groups can be deleted at the same time by shift-clicking the range of groups or alt-clicking (command-clicking on macOS) a non-contiguous set. For the second type, be careful when deleting an entire group. That option is available if any selection within the group is highlighted. That is, an entire beta sheet can be deleted by clicking "Delete Sheet" when any strand within that sheet is highlighted.

Some options are shown in the left panel under "Other settings." These are usually options where it is sensible to apply them to many groups. After several groups are selected, either by shift-clicking a range or alt-clicking (command-clicking on macOS), changing a value in "Other settings" will update all the selected groups. If a parameter in "Other settings" is also shown as a column field, the selection editor will update that column to the new value for the selected groups.

An automatic detection option is available for

- Secondary structure annotations
- NCS groups
- TLS groups

by clicking the "Find ..." button in the left panel. If there are options for tweaking the automatic detection algorithm, they will show up in a new dialog window. These can be changed before running the algorithm. When automatic detection is finished, the selection editor will replace any existing selections with the automatically detected ones. Also, the automatic selections will be saved in your project directory.

Graphical Selection

In addition to manually typing in selections with the proper syntax, selections can also be made with the model viewer by clicking on "Model" in the top toolbar.

The model viewer will display the currently highlighted selections in the current panel in the selection editor. In order to use the model viewer to edit a selection

- Highlight the desired selection for editing in the selection editor
- Click on "Edit Selection" in the model viewer
- Make the changes in the model viewer
- In the model viewer, Click "Accept" to keep the changes, "Discard" to keep the original

Clicking atoms in the model viewer will toggle the highlight on them. To change between highlighting the entire residue or individual atoms, pick the appropriate option in the "Select" field.

The "Show" option in "View Controls" can toggle between showing all the atoms or just the backbone. This can simplify selecting residues.

When editing groups that have multiple selections in the same row (e.g. "Base Pairs" or "Stacking Pairs" under "Secondary Structure"), the specific selection can be changed by the "Field" widget in the model viewer.

A range of residues can be selected or deselected by shift-clicking or alt-clicking (option-clicking on macOS), respectively. The model viewer remembers the last residue that was clicked and uses that as the first residue in the range.

Known Issues

- With the model viewer open, highlighting many selections may be slow; close the model viewer for faster performance when highlighting many selections
- When an invalid type is entered (e.g. a letter is provided when the field expects a number), the selection editor will complain one more time after a valid type is entered.
- After clicking "Validate Syntax", the currently selected or most recently selected rows may not be highlighted in red if the syntax is incorrect.
- On macOS, the text in editing mode can be slightly offset in the row

Guessing sequence from a map with `sequence_from_map`

`sequence_from_map.html`

Author(s)

- `sequence_from_map`: Tom Terwilliger

Purpose

The routine `sequence_from_map` will use a map to guess the sequence of a supplied segment or of segments built automatically.

Usage

How `sequence_from_map` works:

Sequence-from-map examines the features in a map at the position where side chains in a model are expected to be located and uses these features to guess the identity of the side chain at each position.

The procedure used by `sequence-from-map` is has several steps:

If map (and no model) is supplied, a model is created by searching for regular secondary structure in the map. First the unique part of the map is identified and cut out to form a new map.

This is done first using the `map_symmetry` to find helical or point-group symmetry in the map. If any is found, a new map is created containing just the unique part of the map. Then (if `improper_symmetry` is set), the new map is examined to see if there is any local symmetry remaining. If so, the unique part is cut out and a new map is created. Then the tool `find_helices_strands` is used to analyze the new map to find regular secondary structure.

Once a map and model are available, each segment (a piece of chain without breaks) is compared to the map and sequences compatible with the map are identified.

The core of `sequence-from-map` is to generate a scoring matrix for each segment that reflects the relative probability that each possible side chain is located at each position in the segment. This probability is estimated from the map-model correlation for each side chain (after examining all rotamers of the side chain and picking the one with the highest correlation). A Z-score (value for this side chain minus mean for this side chain at all positions, divided by the standard deviation of this quantity) is calculated for each side chain at each position, where Z values less than zero are set to zero. These Z-scores are used as approximate minus-log-likelihood scores for side chain probabilities and make up the scoring matrix for the segment.

A best-fitting sequence is generated for each segment by simply picking the highest-scoring side chain at each position. Optionally, if `score_by_residue_groups` is set and residue groups are defined (`residue_groups = "VGASCTI P LDNEQM KR FHY W"`), one representative of each residue group is chosen to represent all the side chains in that group.

Optionally (if `minimum_discrimination` is defined and `trim_models` is set), the scoring matrix is used to cut the model up into pieces, removing any residues where the discrimination between the most probable and least probable side chain is less than `minimum_discrimination`. The segments are then cut up at these points and smaller segments are created.

Also optionally (if `trim_models` is set to some value N), the N residues on

each end of each segment are removed before analysis.

If one or more potential sequences representing the molecule in the map are provided, these sequences are scored, one at a time, by evaluating the optimal alignment of the sequence to the segments in the model.

The alignment of a sequence to a set of segments is done by considering all possible alignments for all segments, keeping the highest-scoring segment-sequence alignment, then repeating the process with all remaining segments. Optionally, the alignments can be restricted to a set that uses each residue in the sequence only once (`allow_duplicates` set to `False`). Optionally, sequences that are shorter than the number of residues in the model are rejected (if `skip_if_too_short` is set).

The raw score for one segment-sequence alignment is given by the sum of the position-dependent minus-log-likelihood scores for all the side chains in the alignment. This raw score is then adjusted by subtracting the mean value of raw segment-sequence alignment scores for random sequences of the same length as the sequence being considered. The random sequences are created using a residue frequency of eukaryotic amino acids (this can be adjusted with the `default_sequence` parameter). If `positive_only` is set, negative segment-sequence alignment scores are ignored.

Once the best-fitting sequence is identified, the side chains of the model are re-fit using this sequence and a model with the fitted side chains is written out.

Examples

Standard run of `sequence_from_map`:

You can use `sequence_from_map` to guess the sequence of a fragment you have built:

```
phenix.sequence_from_map my_fragment.pdb my_map.mrc resolution=2.8
```

You can also use it to try and pick out which sequence file matches your map:

```
phenix.sequence_from_map my_map.mrc resolution=3 seq_dir=sequences/
```

where the directory `sequences` has a set of fasta or similar files representing possible sequences that might match your map.

Re-sequencing and filling in a gap with `run_assign_sequence=True`:

You can use the special keyword `run_assign_sequence=True` in `sequence_from_map` to carry out a two-step process to try and fix a mistraced part of your model. If residues 154-162 of your model look bad and perhaps the sequence register is wrong after residue 162, you can try something like this:

First remove the bad residues:

```
phenix.pdbtools bad_model.pdb
```

```
remove="resseq 154:162" output.file_name=bad_model_del.pdb
```

Then run `sequence_from_map` to reassign the sequence and fill in the gap (you can supply a `map_file` or an `mtz_file`; either will be fine):

```
phenix.sequence_from_map run_assign_sequence=True
```

```
model_file=bad_model_del.pdb seq_file=sequence.dat map_file=map_file.ccp4
```

```
pdb_out=bad_model_fixed.pdb resolution=2
```

This will produce bad_model_fixed.pdb

Possible Problems

Specific limitations and problems:

Sequence_from_map does not recognize insertion codes, so if your model uses these, first run phenix.pdb_tools with the "renumber=True" keyword to remove them (actually to renumber the residues and leave the insertion codes in place).

Literature

Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003). Rapid model building of alpha-helices in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 268-75 (2010). Rapid model building of beta-sheets in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 276-84 (2010). Protein identification from electron cryomicroscopy maps by automated model-building and side-chain matching. T.C. Terwilliger, O.V. Sobolev, Afonine, PV, P.D. Adams, Ho, CM, Li, X, Zhou, and ZH. Acta Cryst D. 77, 457-462 (2021).

- Automated side-chain model building and sequence assignment by template matching. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 59, 45-9 (2003).
- Rapid model building of alpha-helices in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 268-75 (2010).
- Rapid model building of beta-sheets in electron-density maps. T.C. Terwilliger. Acta Crystallogr D Biol Crystallogr 66, 276-84 (2010).
- Protein identification from electron cryomicroscopy maps by automated model-building and side-chain matching. T.C. Terwilliger, O.V. Sobolev, Afonine, PV, P.D. Adams, Ho, CM, Li, X, Zhou, and ZH. Acta Cryst D. 77, 457-462 (2021).

Additional information

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command line
input_filesmap_file = None File with CCP4-style map. May have origin in any location.
multiple_seq_file = None Optional sequence file containing multiple potential sequences. (Fasta format or sequences separated by blank lines)
seq_file = None Optional sequence file(s) containing potential sequences
seq_dir = None Optional directory containing sequence files
model_file = None Input PDB file with chains to be sequenced NOTE: if you supply model_file no model-building will be done. If you supply starting_model_file it will be used as a starting model for model-building.
starting_model_file = None Input PDB file to be used as starting model in model-building. NOTE: if you supply model_file no model-building will be done. If you supply starting_model_file it will be used as a starting model for model-building.
symmetry_file = None Optional symmetry file (.ncs_spec format or MATRX records) with reconstruction symmetry. Used in identification of unique part of map.
probability_matrix_file = None Optional .pkl file with probability matrices. Note: if build_type is all then a prefix of either helices_strands or full_model will be added to the file name (if not already there)
model_pickle_file = None Optional .pkl file with final fragments. Note: if build_type is all then a prefix of either helices_strands or full_model will be added to the file name (if not already there)
output_files
pdb_out =

sequence_from_map.pdb Model with sequence determined from map. Note: if build_type is all then a prefix of either helices_strands or full_model will be added to the file name (if not already there)output_probability_matrix_file = None Optional output .pkl file with probability matricesoutput_model_pickle_file = None Optional output .pkl file with final fragmentstemp_dir = None Temporary directory. Default is sequence_from_map_xx where xx creates new directorycrystal_inforesolution = None High-resolution limit for map analysis.scattering_table = n_gaussian wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is n_gaussian, for cryoEM is electron.sequences = None Sequences to be matched as text stringsdefault_sequence = " Sequence reflecting relative usage of residues/nucleotides. Default is amino acid composition for eukaryotes.chain_type = *PROTEIN DNA RNA Chain typemolecular_mass = None Molecular mass of the entire structure in the map (not the mass of one chain). Does not have to be accurate, but helpful if using symmetry to extract unique part of map.strategyfill_in_sequence = None Just fill in optimal sequence. Default if no sequence supplied.extract_best_parts = False Try to identify best-fitting parts of sequence and use thosecreate_gap_length = None Create gaps of this length if segments are nearly overlappingextract_length = 35 Target lengths of extracted best parts.extract_cc = 0.80 Target map CC of extracted best parts.extract_cc_delta = 0.01 Delta for target map CC of extracted best parts.extract_cc_min = 0.70 Minimum target map CC of extracted best parts.extract_n = 10 Maximum number of extracted best parts.extract_length_vs_cc = 10 Trade off between length and CCscore_with_optimal_sequence = True Identify optimal sequence with reduced residue set and use it to match sequenceunique_sequence = True Only allow one use of each part of sequence unless model is longer than sequencesplit_using_alignment = None At end, split fragments based on alignment. Default is True if score_with_optimal_sequence is True and sequence is suppliedkeep_segment_order = None Keep order of segments (only change their local positions). Only applies if unique_sequence=True and score_with_optimal_sequence=Trueextract_ss = False Extract secondary structure groups (usual alternative is extract regions)extract_ss_dist_max = 15 RMSD of ends of secondary structure must match within this dist.extract_ss_groups = 1 Number of secondary structure groups t...

- job_title = None Job title in PHENIX GUI, not used on command line
- input_filesmap_file = None File with CCP4-style map. May have origin in any location.multiple_seq_file = None Optional sequence file containing multiple potential sequences. (Fasta format or sequences separated by blank lines)seq_file = None Optional sequence file(s) containing potential sequencesseq_dir = None Optional directory containing sequence filesmodel_file = None Input PDB file with chains to be sequenced NOTE: if you supply model_file no model-building will be done. If you supply starting_model_file it will be used as a starting model for model-building.starting_model_file = None Input PDB file to be used as starting model in model-building. NOTE: if you supply model_file no model-building will be done. If you supply starting_model_file it will be used as a starting model for model-building.symmetry_file = None Optional symmetry file (.ncs_spec format or MATRX records) with reconstruction symmetry. Used in identification of unique part of map.probability_matrix_file = None Optional .pkl file with probability matrices. Note: if build_type is all then a prefix of either helices_strands or full_model will be added to the file name (if not already there)model_pickle_file = None Optional .pkl file with final fragments. Note: if build_type is all then a prefix of either helices_strands or full_model will be added to the file name (if not already there)
- map_file = None File with CCP4-style map. May have origin in any location.
- multiple_seq_file = None Optional sequence file containing multiple potential sequences. (Fasta format or sequences separated by blank lines)
- seq_file = None Optional sequence file(s) containing potential sequences
- seq_dir = None Optional directory containing sequence files

- `model_file` = None Input PDB file with chains to be sequenced NOTE: if you supply `model_file` no model-building will be done. If you supply `starting_model_file` it will be used as a starting model for model-building.
- `starting_model_file` = None Input PDB file to be used as starting model in model-building. NOTE: if you supply `model_file` no model-building will be done. If you supply `starting_model_file` it will be used as a starting model for model-building.
- `symmetry_file` = None Optional symmetry file (.ncs_spec format or MATRX records) with reconstruction symmetry. Used in identification of unique part of map.
- `probability_matrix_file` = None Optional .pkl file with probability matrices. Note: if `build_type` is all then a prefix of either `helices_strands` or `full_model` will be added to the file name (if not already there)
- `model_pickle_file` = None Optional .pkl file with final fragments. Note: if `build_type` is all then a prefix of either `helices_strands` or `full_model` will be added to the file name (if not already there)
- `output_files` `pdb_out` = `sequence_from_map.pdb` Model with sequence determined from map. Note: if `build_type` is all then a prefix of either `helices_strands` or `full_model` will be added to the file name (if not already there) `output_probability_matrix_file` = None Optional output .pkl file with probability matrices `output_model_pickle_file` = None Optional output .pkl file with final fragments `temp_dir` = None Temporary directory. Default is `sequence_from_map_xx` where `xx` creates new directory
- `pdb_out` = `sequence_from_map.pdb` Model with sequence determined from map. Note: if `build_type` is all then a prefix of either `helices_strands` or `full_model` will be added to the file name (if not already there)
- `output_probability_matrix_file` = None Optional output .pkl file with probability matrices
- `output_model_pickle_file` = None Optional output .pkl file with final fragments
- `temp_dir` = None Temporary directory. Default is `sequence_from_map_xx` where `xx` creates new directory
- `crystal_info` `resolution` = None High-resolution limit for map analysis. `scattering_table` = `n_gaussian` wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is `n_gaussian`, for cryoEM is `electron`. `sequences` = None Sequences to be matched as text strings `default_sequence` = " Sequence reflecting relative usage of residues/nucleotides. Default is amino acid composition for eukaryotes. `chain_type` = *PROTEIN DNA RNA Chain type `molecular_mass` = None Molecular mass of the entire structure in the map (not the mass of one chain). Does not have to be accurate, but helpful if using symmetry to extract unique part of map.
- `resolution` = None High-resolution limit for map analysis.
- `scattering_table` = `n_gaussian` wk1995 it1992 *electron neutron Choice of scattering table for structure factor calculations. Standard for X-ray is `n_gaussian`, for cryoEM is `electron`.
- `sequences` = None Sequences to be matched as text strings
- `default_sequence` = " Sequence reflecting relative usage of residues/nucleotides. Default is amino acid composition for eukaryotes.
- `chain_type` = *PROTEIN DNA RNA Chain type
- `molecular_mass` = None Molecular mass of the entire structure in the map (not the mass of one chain). Does not have to be accurate, but helpful if using symmetry to extract unique part of map.
- `strategy` `fill_in_sequence` = None Just fill in optimal sequence. Default if no sequence supplied. `extract_best_parts` = False Try to identify best-fitting parts of sequence and use those `create_gap_length` = None Create gaps of this length if segments are nearly overlapping `extract_length` = 35 Target lengths of extracted best parts. `extract_cc` = 0.80 Target map CC

of extracted best parts.
`extract_cc_delta = 0.01` Delta for target map CC of extracted best parts.
`extract_cc_min = 0.70` Minimum target map CC of extracted best parts.
`extract_n = 10` Maximum number of extracted best parts.
`extract_length_vs_cc = 10` Trade off between length and CC
`score_with_optimal_sequence = True` Identify optimal sequence with reduced residue set and use it to match sequence
`unique_sequence = True` Only allow one use of each part of sequence unless model is longer than sequence
`split_using_alignment = None` At end, split fragments based on alignment. Default is True if `score_with_optimal_sequence` is True and sequence is supplied
`keep_segment_order = None` Keep order of segments (only change their local positions). Only applies if `unique_sequence=True` and `score_with_optimal_sequence=True`
`extract_ss = False` Extract secondary structure groups (usual alternative is extract regions)
`extract_ss_dist_max = 15` RMSD of ends of secondary structure must match within this dist.
`extract_ss_groups = 1` Number of secondary structure groups to generate
`build_only = False` Just set up and build model and stop
`retrace = None` Retrace chains and score with map correlation and sequence alignment in trace_and_build procedure. Default is False if quick = True
`block_gaps_in_secondary_structure = True` Block creation of gaps in alignments in the middle of secondary structure
`run_map_box = False` Run map box on input model with radius
`atom_radiusmap_box_radius = 10` RMSD of ends of secondary structure must match within this dist.
`rerun_with_avail_seq = True` Rerun sequence alignment with parts of the sequence already used removed. With a large structure this can take a long time.
`speed_up_ca_insertion = False` Speed up CA insertion by running on short segments first.
`chains_from_fixed_model = None` Use fixed model to mark initial chain positions.
`max_fraction_long_branches = None` Terminate trace if recent fraction of long branches is high
`split_and_join_iterations = None` Iterations of split_and_join
`split_and_join = None` Split fragments at low density and then rejoin. Criterion is `sd_ratio_pare_model`
`rejoin_split_fragments = False` Specifically rejoin the split fragments leaving out poor junctions
`test_threshold_fixed_segments = None` Test thresholds when creating fixed segments
`require_ends_in_region = None` Add ends of chain to region if necessary
`try_alternate_joins = None` try joining each end of pair of fragments, not just the first
`skip_path_if_missing = None` Skip path if points are missing

- `fill_in_sequence = None` Just fill in optimal sequence. Default if no sequence supplied.
- `extract_best_parts = False` Try to identify best-fitting parts of sequence and use those
- `create_gap_length = None` Create gaps of this length if segments are nearly overlapping
- `extract_length = 35` Target lengths of extracted best parts.
- `extract_cc = 0.80` Target map CC of extracted best parts.
- `extract_cc_delta = 0.01` Delta for target map CC of extracted best parts.
- `extract_cc_min = 0.70` Minimum target map CC of extracted best parts.
- `extract_n = 10` Maximum number of extracted best parts.
- `extract_length_vs_cc = 10` Trade off between length and CC
- `score_with_optimal_sequence = True` Identify optimal sequence with reduced residue set and use it to match sequence
- `unique_sequence = True` Only allow one use of each part of sequence unless model is longer than sequence
- `split_using_alignment = None` At end, split fragments based on alignment. Default is True if `score_with_optimal_sequence` is True and sequence is supplied
- `keep_segment_order = None` Keep order of segments (only change their local positions). Only applies if `unique_sequence=True` and `score_with_optimal_sequence=True`
- `extract_ss = False` Extract secondary structure groups (usual alternative is extract regions)

- `extract_ss_dist_max` = 15 RMSD of ends of secondary structure must match within this dist.
- `extract_ss_groups` = 1 Number of secondary structure groups to generate
- `build_only` = False Just set up and build model and stop
- `retrace` = None Retrace chains and score with map correlation and sequence alignment in `trace_and_build` procedure. Default is False if `quick` = True
- `block_gaps_in_secondary_structure` = True Block creation of gaps in alignments in the middle of secondary structure
- `run_map_box` = False Run map box on input model with `radius_atom_radius`
- `map_box_radius` = 10 RMSD of ends of secondary structure must match within this dist.
- `rerun_with_avail_seq` = True Rerun sequence alignment with parts of the sequence already used removed. With a large structure this can take a long time.
- `speed_up_ca_insertion` = False Speed up CA insertion by running on short segments first.
- `chains_from_fixed_model` = None Use fixed model to mark initial chain positions.
- `max_fraction_long_branches` = None Terminate trace if recent fraction of long branches is high
- `split_and_join_iterations` = None Iterations of `split_and_join`
- `split_and_join` = None Split fragments at low density and then rejoin. Criterion is `sd_ratio_pare_model`
- `rejoin_split_fragments` = False Specifically rejoin the split fragments leaving out poor junctions
- `test_threshold_fixed_segments` = None Test thresholds when creating fixed segments
- `require_ends_in_region` = None Add ends of chain to region if necessary
- `try_alterate_joins` = None try joining each end of pair of fragments, not just the first
- `skip_path_if_missing` = None Skip path if points are missing
- `sequencingcreate_final_model` = True Create final model with sequence
- `obtainedresidue_number_from_sequence` = True Start chains with best guess of residue number from sequence file
- `build_type` = *None helices_and_full helices_strands full_model trace_and_build trace_and_build_two_cycles Default is trace_and_build if `quick` = true and trace_and_build_two_cycles if `quick` = False. You can choose to build with trace_and_build, just helices/strands (quick) or a full model (takes more time). Note you can define how many regions to analyze with `regions_to_keep` (set it to 1 to run quickly). You can choose helices_and_full to run both helices_strands and full_model or trace_and_build_two_cycles to run trace and build two cycles and pick the best result.
- `tnb_build_cycles` = 1 Trace and build build cycles
- `regions_to_keep` = None You can specify a limit to the number of regions to keep when generating the asymmetric unit of density. Default is 1 for `build_type` = full_model or trace_and_build and None for `build_type` = helices_strands.
- `minimum_score` = -10. Minimum score. Scores less than this will be replaced with this value.
- `alignment_method` = *smith_waterman muscle Alignment method. Alternatives are muscle and smith_waterman
- `gap_ratio` = 0.5 Ratio of gap extension penalty to gap opening penalty
- `blocking_ratio` = 10 Ratio of gap extension penalty in regions to be blocked to to allowed regions
- `gap_opening_penalty_max` = None Gap opening penalty for sequence alignments. Goes from `gap_opening_penalty_min` to `gap_opening_penalty_max`. Default is 10 for muscle and 150 for phenix
- `gap_opening_penalty_min` = None Gap opening penalty for sequence alignments. Goes from `gap_opening_penalty_min` to `gap_opening_penalty_max`. Default is 2.5 for muscle and 35 for phenix
- `trim_ends_by` = 1 When blocking alignments with secondary structure, trim the secondary structure back by this amount to allow insertions and deletions at the end of the secondary structure.
- `convincing_delta_score` = 10. Convincing delta score for two solutions that nearly always means the higher-scoring one is correct.
- `average_z_scores` = False Use average Z-score for multiple builds.

Alternative is use highest score.
`minimum_final_z_score = 4` Skip additional builds if a solution with this Z-score or better is found.
`minimum_length_after_split = 5` Minimum length of a segment after splitting with `split_using_alignment`
`minimum_length_after_split_penalty = 2` Penalty per residue shorter than `minimum_length`
`minimum_length = None` Minimum length of a segment to consider it for sequencing. Default is 10 if a `multiple_sequence_file` or `sequence_dir` is supplied
`minimum_length_ratio = None` Reject segments that are less than this fraction of longest segment. Applies if `max_segments` is not set. Default is 0.15 if a `multiple_sequence_file` or `sequence_dir` is supplied.
`maximum_segments_in_sequencing = None` Maximum number of segments to consider (remainder ignored) in analyzing sequence placement. Default is 10 if a `multiple_sequence_file` or `sequence_dir` is supplied.
`max_segments = None` Maximum segments to consider in analyzing sequences. Default is to take all segments unless a `multiple_sequence_file` or `sequence_dir` is supplied, in which case the default is 3.
`minimum_residues = None` Try to choose enough segments to get this many residues. Applies if `max_segments` is not set. Default is 500 if multiple sequences are supplied.
`maximum_sequence_length = None` Maximum sequence length to consider. Default is 2000 if multiple sequences are supplied.
`subtract_expected = None` *single_try multiple_tries Subtract expected score based on random sequence for a single try or for the number of tries carried out.
`positive_only = False` Just sum scores that are positive and skip others
`allow_duplicates = True` Allow duplication of use of sequence.
`skip_if_too_short = False` Skip sequences that are shorter than the number of residues in the model.
`skip_if_too_short_for_segment = None` Skip sequences that are shorter than the number of residues in a segment, with tolerance of...

- `create_final_model = True` Create final model with sequence obtained
- `residue_number_from_sequence = True` Start chains with best guess of residue number from sequence file
- `build_type = *None helices_and_full helices_strands full_model trace_and_build trace_and_build_two_cycles` Default is `trace_and_build` if `quick = True` and `trace_and_build_two_cycles` if `quick = False`. You can choose to build with `trace_and_build`, just helices/strands (`quick`) or a full model (takes more time). Note you can define how many regions to analyze with `regions_to_keep` (set it to 1 to run quickly). You can choose `helices_and_full` to run both `helices_strands` and `full_model` or `trace_and_build_two_cycles` to run trace and build two cycles and pick the best result.
- `tnb_build_cycles = 1` Trace and build build cycles
- `regions_to_keep = None` You can specify a limit to the number of regions to keep when generating the asymmetric unit of density. Default is 1 for `build_type = full_model` or `trace_and_build` and `None` for `build_type = helices_strands`.
- `minimum_score = -10`. Minimum score. Scores less than this will be replaced with this value.
- `alignment_method = *smith_waterman muscle` Alignment method. Alternatives are `muscle` and `smith_waterman`
- `gap_ratio = 0.5` Ratio of gap extension penalty to gap opening penalty
- `blocking_ratio = 10` Ratio of gap extension penalty in regions to be blocked to to allowed regions
- `gap_opening_penalty_max = None` Gap opening penalty for sequence alignments. Goes from `gap_opening_penalty_min` to `gap_opening_penalty_max`. Default is 10 for `muscle` and 150 for `phenix`
- `gap_opening_penalty_min = None` Gap opening penalty for sequence alignments. Goes from `gap_opening_penalty_min` to `gap_opening_penalty_max`. Default is 2.5 for `muscle` and 35 for `phenix`
- `trim_ends_by = 1` When blocking alignments with secondary structure, trim the secondary structure back by this amount to allow insertions and deletions at the end of the secondary structure.

- `convincing_delta_score = 10`. Convincing delta score for two solutions that nearly always means the higher-scoring one is correct.
- `average_z_scores = False` Use average Z-score for multiple builds. Alternative is use highest score.
- `minimum_final_z_score = 4` Skip additional builds if a solution with this Z-score or better is found.
- `minimum_length_after_split = 5` Minimum length of a segment after splitting with `split_using_alignment`
- `minimum_length_after_split_penalty = 2` Penalty per residue shorter than minimum
- `minimum_length = None` Minimum length of a segment to consider it for sequencing. Default is 10 if a `multiple_sequence_file` or `sequence_dir` is supplied
- `minimum_length_ratio = None` Reject segments that are less than this fraction of longest segment. Applies if `max_segments` is not set. Default is 0.15 if a `multiple_sequence_file` or `sequence_dir` is supplied.
- `maximum_segments_in_sequencing = None` Maximum number of segments to consider (remainder ignored) in analyzing sequence placement. Default is 10 if a `multiple_sequence_file` or `sequence_dir` is supplied.
- `max_segments = None` Maximum segments to consider in analyzing sequences. Default is to take all segments unless a `multiple_sequence_file` or `sequence_dir` is supplied, in which case the default is 3.
- `minimum_residues = None` Try to choose enough segments to get this many residues. Applies if `max_segments` is not set. Default is 500 if multiple sequences are supplied.
- `maximum_sequence_length = None` Maximum sequence length to consider. Default is 2000 if multiple sequences are supplied.
- `subtract_expected = None` *single_try multiple_tries Subtract expected score based on random sequence for a single try or for the number of tries carried out.
- `positive_only = False` Just sum scores that are positive and skip others
- `allow_duplicates = True` Allow duplication of use of sequence.
- `skip_if_too_short = False` Skip sequences that are shorter than the number of residues in the model.
- `skip_if_too_short_for_segment = None` Skip sequences that are shorter than the number of residues in a segment, with tolerance of `short_tolerance_percent`.
- `short_tolerance_percent = 10` Tolerance for `skip_if_too_short_for_segment`
- `refinement = None` Run refinement after finding model. Default is True for `helices_strands` or `full_model` and False for `trace_and_build`.
- `simulated_annealing_tries = None` Run simulated annealing this many times on segments and weight sequence probability based on the resulting likelihood scores.
- `average_sa_by_residue = False` Average SA tries by residue (alternative is average by model)
- `split_then_fix = False` First use `split_at_unknown` to split up model, screen all input sequences and choose top `n_split_then_fix` sequences. Then run again on these sequences using `fix_insertions_deletions` on each of them for final scoring.
- `n_split_then_fix = None` Number of sequences to keep in play in `split_then_fix`
- `fix_insertions_deletions = False` Try to fix alignment allowing up to $n_split * n_sites / 10$ pieces but keep them linked to maximize sequence match
- `n_split = 1` Maximum number of pieces to cut segments up into is $n_sites * n_split / 10$

- `max_start_points_to_consider = 5` Maximum number of starting points for sequence to consider per 100 residues in sequence file.
- `positive_gap_penalty = 1` Penalty for missing residues is `positive_gap_penalty * gap**2`
- `negative_gap_penalty = 2` Gap penalty for extra residues is `negative_gap_penalty * gap**2`
- `max_gap_length = 2` Maximum gap length for adjacent alignments
- `end_overlap = 2` Maximum end overlap of fragments
- `minimum_alignment_length = 5` Minimum length of an alignment
- `test_lengths = 10 20` Lengths of segments to be tested for sequence matching.
- `macro_cycles = 5` Refinement macro-cycles
- `trim_from_ends = None` You can take final fragments and trim them N residues in from each end.
- `residue_groups = "GVASCTI P LDNEQM KR YFH W"` Optional groups of residues to score together. If RNA or DNA groups are not allowed.
- `score_by_residue_groups = None` Use residue groups in sequence alignment and listing of optimal sequences. Default is True unless no sequence is supplied.
- `trim_models = False` Trim models to break at residues with low discrimination. Default is False.
- `maximum_length = 50` If segment is longer than this, try to cut it into pieces, splitting at residues that are poorly-defined. Assumption is very long segments probably have register shifts. Only applies if `trim_models = True`
- `split_at_unknown = None` Split chains at positions where residue is unknown (X). Default is True unless `fix_insertions_deletions = True` or `score_with_optimal_sequence = True`
- `keep_split_together = True` Apply penalty if chains that are split at unknown are not close in sequence
- `vary_minimum_cc = False` Vary the minimum_cc in optimizing percent to reject. Alternative is `vary_minimum_z_score`.
- `percent_to_reject = 7.5` Adjust minimum_cc and minimum_z_score to reject this percentage of residues.
- `minimum_z_score = 0.5` Minimum residue Z-score for a residue (or residue groups if `score_by_residue_groups` is set) to keep the choice. Otherwise report residue as X.
- `minimum_cc = None` Minimum map-model correlation for a residue (or residue groups if `score_by_residue_groups` is set) to keep the choice. Otherwise report residue as X.
- `minimum_discrimination = 0.01` Minimum discrimination between lowest and highest probability residue (or residue groups if `score_by_residue_groups` is set) to keep the choice. Otherwise report residue as X.
- `random_sequences = 20` Number of random sequences of each length to use as baseline
- `use_default_sequence = False` Base the random sequences on the default sequence. Alternative is to base them on the actual sequence
- `remove_clashes = True` Truncate clashing side chains on output
- `symmetry = ANY` Optional symmetry used in reconstruction. For example D7, C3, C2 I (icosahedral), T (tetrahedral), or ANY (try everything and use the highest symmetry found). Not needed if symmetry is supplied.
- `include_helical_symmetry = False` Include search for helical symmetry if `symmetry = ALL`
- `run_assign_sequence = False` Run assign sequence to identify sequence of multiple fragments and connect them.

- `improper_symmetry = False` After any proper symmetry is found, take new unique part of map and search for any symmetry at all with `find_ncs_from_density`. Useful if your molecule has both proper reconstruction symmetry such as C3 and within the reconstruction unit there is local symmetry which can be of any kind. Final result is just the unique part of the map.
- `separate_chains = True` Write each segment as a separate chain with unique chain ID
- `null_score = -2` Score for a segment that is not placed
- `n_direct = 10` Extreme values for $N \leq n_direct$ calculated directly
- `distance_cutoff = None` Distance cutoff (CA-CA or P-P) for chain breaks. Large value used to prevent breaking chain due to big gaps.
- `min_ncs_cc = 0.80` Minimum Symmetry CC to keep operators when identifying automatically
- `rebuildingsplit_input_model = True` Input model will be split into segments
- `refine = True` Refine all-atom models at end of procedure
- `refine_b = None` Refine B-values
- `refine_cycles = 1` Refinement cycles (except final cycles). Typical is 1 for quick or superquick and 5 for thorough building
- `trace_and_build_min_segment_length = 15` Minimum segment length in `trace_and_build`
- `controlmultiprocessing = *` `multiprocessing` `sgs` `lsf` `pbs` `condor` `pbspro` `slurm` Choices are multiprocessing (single machine) or queuing systems
- `queue_run_command = None` run command for queue jobs. For example `qsub`.
- `nproc = 1` Number of processors to use
- `random_seed = 171731` Random seed
- `verbose = False` Verbose output
- `quick = True` Quick run. Suitable for most cases. Use `quick = False` to obtain maximum sensitivity by setting `build_type = trace_and_build_two_cycles`
- `simulated_annealing_tries = 32`
- `superquick = False` Very quick run
- `ignore_symmetry_conflicts = False` You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with `box_map` for example.
- `max_dirs = 1000` Maximum number of directories (sequence_from_map_xxx)
- `em_side_density = False` Use EM side chain density. Alternative is to use standard x-ray side chain templates in sequence assignment
- `em_side_density_in_tnb = False` EM side chain density in `trace_and_build` (if `em_side_density` is True)
- `skip_temp_dir = False` Do not create or use a temp_dir
- `clean_up = False` Clean up and remove temporary sequence_from_map directory after finishing

- `ignore_symmetry_conflicts = False` You can ignore the symmetry information (CRYST1) from coordinate files. This may be necessary if your model has been placed in a box with `box_map` for example.
- `max_dirs = 1000` Maximum number of directories (`sequence_from_map_xxx`)
- `em_side_density = False` Use EM side chain density. Alternative is to use standard x-ray side chain templates in sequence assignment
- `em_side_density_in_tnb = False` EM side chain density in `trace_and_build` (if `em_side_density` is True)
- `skip_temp_dir = False` Do not create or use a `temp_dir`
- `clean_up = False` Clean up and remove temporary `sequence_from_map` directory after finishing
- guiGUI-specific parameter required for output directory `output_dir = None`
- `output_dir = None`

Setting up tests that can run automatically in the Phenix GUI

set_up_gui_tests.html

Author(s)

- Automatic GUI tests: Tom Terwilliger

Purpose

These tests can be used to make sure that programs in the GUI can run properly. They can also be used as an introduction to running in the GUI as they are all preset to run.

Framework

The GUI_TESTS directory has a set of subdirectories (CRYSTALS_A...) that can be imported as projects into the Phenix GUI. Once in the GUI, all the unique jobs in a project can be run sequentially with the "Auto-run all unique jobs" function.

Updating the GUI tests

New projects can be added to the GUI_TESTS directory by creating them in the Phenix GUI in an installation of GUI_TESTS. The existing and new projects can then be packaged into a new version of GUI_TESTS automatically. Additionally, new jobs can be run in an existing GUI_TESTS module and existing jobs can be deleted. On repackaging, the GUI_TESTS will reflect the state of these tests in the GUI that was worked on.

How to set up the tests

The tests are supplied with Phenix, but you have to install them and import them into the GUI to use them. Here is how:

Install the tests

Decide where you want to put the GUI_TESTS (in this example, ~/Downloads)

Go to the phenix_regression directory with GUI_TESTS (i.e.,

\$PHENIX/modules/phenix_regression/gui/auto_run). If \$PHENIX is not defined, you will need to type in the full path.

```
cd $PHENIX/modules/phenix_regression/gui/auto_run
```

To install in ~/Downloads, type:

```
./install_gui_tests.sh ~/Downloads
```

This will install the GUI_TESTS directory with the right path names in

~/Downloads.

Start the Phenix GUI. You might want to do this from the command line instead of clicking an icon so that the log goes to the terminal window.

In the Phenix GUI: select "Import Project". Navigate to your new GUI_TESTS directory, then to each subdirectory (CRYSTALS_A CRYSTALS_B, etc) and import each one individually.

- In the Phenix GUI: select "Import Project".
- Navigate to your new GUI_TESTS directory, then to each subdirectory (CRYSTALS_A CRYSTALS_B, etc) and import each one individually.

Navigate to your new GUI_TESTS directory, then to each subdirectory (CRYSTALS_A CRYSTALS_B, etc) and import each one individually.

Run a test

You are now ready to run one or all the tests. To run just one test:

In the Phenix GUI, select one of these projects (MODEL_A) and go to Job History. Select any job and hit "restore" or double click it The job will be restored and you can run it with "Run"

- In the Phenix GUI, select one of these projects (MODEL_A) and go to Job History.
- Select any job and hit "restore" or double click it
- The job will be restored and you can run it with "Run"

Run a group of tests

In the Phenix GUI, select one of these projects (MODEL_A) and go to Job History. Hit "Auto-run all unique jobs" The jobs will be run sequentially. Look for any errors. A few jobs will require clicking "OK" to go on. (It will stop and wait for your OK). It will also stop on most errors. Tests that cannot be run this way are in the ONE_BY_ONE project. For these, just reload the job and run it.

- In the Phenix GUI, select one of these projects (MODEL_A) and go to Job History.
- Hit "Auto-run all unique jobs"
- The jobs will be run sequentially. Look for any errors. A few jobs will require clicking "OK" to go on. (It will stop and wait for your OK). It will also stop on most errors. Tests that cannot be run this way are in the ONE_BY_ONE project. For these, just reload the job and run it.

The jobs will be run sequentially. Look for any errors.

A few jobs will require clicking "OK" to go on. (It will stop and wait for your OK). It will also stop on most errors. Tests that cannot be run this way are in the ONE_BY_ONE project. For these, just reload the job and run it.

Modifying the GUI_TESTS jobs

You can modify the GUI_TESTS jobs in any way and save the new version in phenix_regression. Details below. Basically: install the GUI_TESTS, create new jobs, delete any jobs that you do not want saved, create new projects (must be subdirectories of the directory GUI_TESTS), whatever. Then package up the new version you have created with the instructions below, and commit it to the repository.

Details of modifying the GUI_TESTS jobs by creating a new set of tests

When you have a new installation that you would like to save, you can use the create_new_gui_tests_package.csh script in your GUI_TESTS directory. NOTE: Script is c-shell so it won't work on bash for now.

- Get into your GUI_TESTS directory:

```
cd GUI_TESTS
```

2. Decide where your new installation package goes. The instructions here will put it in your ~/Downloads directory and you can copy it to your regression phenix_regression/gui/auto_run directory:

- Create a new installation package:

```
./create_new_gui_tests_package.csh ~/Downloads
```

```
cp ~/Downloads/GUI_TESTS.tgz $PHENIX/modules/phenix_regression/gui/auto_run/
```

This will overwrite the GUI_TESTS.tgz package in Downloads, and if you copy it, the one in your regression directory. You can put your new installation package anywhere you like.

Showing information about a map file with show_map_info

show_map_info.html

Author(s)

- show_map_info: Tom Terwilliger

Purpose

The routine show_map_info will show a summary of the data in a map file (ccp4/mrc/map).

Definitions of map specifications

A map file contains a box of data on a grid. This box of data may be the original box of data or it might be a smaller box cut out of the original box of data.

The map file also contains meta-data including a 'unit cell' and gridding for a unit cell. The unit cell usually corresponds to the full original box of data. The 'unit cell grid' is the grid corresponding to the unit cell. The unit cell has dimensions defined by 'unit cell parameters'.

The map that is present is represented on part or all of the unit cell grid. It starts at the grid point called the 'origin' and goes to one grid point in each direction before the grid point called 'last'. The map that is present has 'all' grid points in each direction. The map that is present has a 'map unit cell' and 'map grid' that correspond to the dimensions 'all'.

Example: a full map with a unit cell grid of (10, 10, 10) goes from (0, 0, 0) to (9, 9, 9) and has dimensions of (10, 10, 10).

Example: a partial map with a unit cell grid of (10,10,10) and an origin of (0, 0, 2) and a map grid or 'all' of (6, 6, 6), would go from (0, 0, 2) to (5, 5, 7).

A map file has a 'pixel size' in each direction (usually they are all the same). The pixel size is the dimension along x, y, or z divided by the number of grid points along that direction.

A map file has a 'map unit cell' which corresponds to the dimensions of the box of data that is present in the map. This will be the same as the unit cell of the full map if the entire map is present.

External origin (ORIGIN record in map file)

A map file can have an 'external_origin' defined. The external origin is an offset that is intended to be applied to a model file to superimpose the model on the map. If a map has an external origin defined, show_map_info will print out this value.

Phenix does not generally support the external_origin values. You can however shift a model file (PDB/mmCIF) based on a value of the external_origin with the tool phenix.shift_model_to_match_map

```
phenix.shift_model_to_match_map map_file.mrc model_file.pdb
```

Wrapping of a map

Maps can be periodic (wrapping = True) or non-periodic (wrapping = False). If you cut out a box from a periodic map, the new map is non-periodic.

If a map is periodic, then values at coordinates (x,y,z) outside the map are defined and can be obtained simply by translating (x,y,z) by integral numbers of the unit cell translations a,b,c ($N * a$, $M * b$, $O * c$) where N,M and O are integers. If a map is not periodic, values outside the supplied map are not defined.

In most Phenix methods, it is assumed that cryo-EM maps are non-periodic and X-ray (crystal) maps are periodic. You can specify in many cases whether your map is periodic or not, however, using the 'wrapping' keyword.

When you run `show_map_info` it will tell you whether the map supports wrapping.

Standard run of `show_map_info`:

You can use `show_map_info` to summarize a map file like this:

```
phenix.show_map_info my_map_file.mrc
```

You can get the value of the map at a particular grid point with:

```
phenix.show_map_info my_map_file.mrc grid_point_to_show=6,2,5
```

Possible Problems

Specific limitations and problems:

Literature

The MRC (ccp4/mrc/map) file format is defined at MRC/CCP4 file format for images and volumes

Additional information

List of all available keywords

`job_title` = None Job title in PHENIX GUI, not used on command line
`input_filesmap_file` = None File with CCP4-style map.
`map_infogrid_point_to_show` = None Display map value at this grid point if set. You can specify more than one.
`n_tries` = 23 Number of tries to find a common value
`relative_sd_tol` = 0.01 Tolerance for constant value at boundaries (relative_sd_tol). Used in identifying wrapping.
`check_wrapping` = None You can explicitly test whether this map appears suitable for wrapping (values outside map mapped inside with unit cell translations).
`gui` GUI-specific parameter required for output directory
`output_dir` = None

- `job_title` = None Job title in PHENIX GUI, not used on command line
- `input_filesmap_file` = None File with CCP4-style map.
- `map_file` = None File with CCP4-style map.
- `map_infogrid_point_to_show` = None Display map value at this grid point if set. You can specify more than one.
- `n_tries` = 23 Number of tries to find a common value
- `relative_sd_tol` = 0.01 Tolerance for constant value at boundaries (relative_sd_tol). Used in identifying wrapping.
- `check_wrapping` = None You can explicitly test whether this map appears suitable for wrapping (values outside map mapped inside with unit cell translations).
- `grid_point_to_show` = None Display map value at this grid point if set. You can specify more than one.
- `n_tries` = 23 Number of tries to find a common value

- `relative_sd_tol = 0.01` Tolerance for constant value at boundaries (`relative_sd_tol`). Used in identifying wrapping.
- `check_wrapping = None` You can explicitly test whether this map appears suitable for wrapping (values outside map mapped inside with unit cell translations).
- `guiGUI-specific parameter required for output directory``output_dir = None`
- `output_dir = None`

Homology modelling using simple_homology_model

simple_homology_model.html

Contents

- Purpose
- Installation
- Usage
- Input files
- Output files
- Description Chain-to-alignment-matching Homology modelling setup Homology modelling
- Chain-to-alignment-matching
- Homology modelling setup
- Homology modelling
- Command line Command-line switches: PHIL arguments:
- Command-line switches:
- PHIL arguments:
- Warning and error messages Error messages Warnings
- Error messages
- Warnings
- Additional information List of all available keywords
- List of all available keywords

Purpose

This program can be used for building a homology model for a target sequence using a template structure and an alignment.

Installation

simple_homology_model uses Rosetta, software developed in the Baker laboratory at the University of Washington. See the central installation notes for Rosetta.

Usage

simple_homology_model is available from the command line.

Input files

- Structure (compulsory). The template structure. Accepted formats: PDB. Recognized extensions: .pdb, .ent.
- Alignment (compulsory). Alignment between the target and template sequences. Accepted formats: .aln: CLUSTAL format. .pir: PIR format. .ali: relaxed PIR-like format.

- .aln: CLUSTAL format.
- .pir: PIR format.
- .ali: relaxed PIR-like format.
- Sequence (optional). The complete target sequence if the alignment does not fully cover it. If not specified, the first sequence from the alignment will be used as target.

Output files

The homology model is written out as specified in output.model.

Description

The program first discard all residues from the template structure that are not present in the target and morphs the residue type when they are (this step is performed by Sculptor and is governed by the alignment). The missing segments are detected, a suitable loop modelling setup is devised, which is then fed into Rosetta to do the actual building. For a summary of all keywords with the corresponding defaults, see the Additional information section.

Chain-to-alignment-matching

This controls how chain-to-alignment matching is performed. For more information, see the Sculptor documentation.

Homology modelling setup

For each loop, the program backtracks min_residue_margin number of residues on both ends, then checks the distance between bridge residues. If this is shorter than residue_distance * number-of-residues-in-between, than backtracks one residues more on each side, and keeps doing this until the criterion is statisfied, or hits a terminus on either end. After this, it evaluates residue segments that are not assigned for loop modelling and will be kept fixed during the modelling step. Segments shorter than min_edge_segment_length will be discarded at the termini, and shorter than min_intenal_segment_length will be discarded internally, and the surrounding loops joined. It then checks the loop lengths, and stops with an error message if a loop is longer than max_loop_length.

Homology modelling

The program runs Rosetta with a kinematic loop closure algorithm (rosetta_loop_closure), as this does not require selected fragments. After the loop is closed, Rosetta can be instructed to run a refinement (rosetta_loop_refinement).

Command line

```
phaser.simple_homology_model \
[ command-line switches ] \
[ PHIL-format parameter files ] \
[ PHIL command-line assignments ] \
[ PDB file ] \
[ alignment file ] \
[ sequence file ]
```

Command-line switches:


```
-h, --help show this help message and exit
--show-defaults print PHIL and exit
-i, --stdin read PHIL from stdin as well
```

PHIL arguments:

Everything not starting with a dash('-') is interpreted as a PHIL argument. This can be a PHIL-format file containing parameters, command-line assignment or a file whose type is automatically recognized (based on file extension; structure files and alignment files are recognized automatically).

Warning and error messages

Error messages

- Structure file is missing: no PDB file specified.
- Chain .. is not recognised as protein: the chain specified for modelling is not protein.
- Alignment file is missing: no alignment file specified.
- Could not match chain with alignment: chain-to-alignment matching was not successful (reason indicated in previous line).
- ... loop longer than maximum allowed: loop .. is longer than the maximum allowed.
- Incomplete Rosetta environment setup: ... Rosetta environment is not setup.
- Cannot locate loopmodel: ..": the ``loopmodel Rosetta executable is not found.
- Cannot locate Rosetta database: the Rosetta database is not found.
- Rosetta finished with error code ...: Rosetta exited with an error code.
- No models produced: although Rosetta exited indicating success, no model is created.

Warnings

- No sequence file provided: no sequence file provided, the first sequence from the alignment will be used as target.
- `` Multiple models output - using only first``: Rosetta generated multiple models, and only the first one will be used.

Additional information

List of all available keywords

inputParameters controlling input
structure = None PDB file name
alignment = None Alignment file
name
sequence = None Target sequence
outputParameters controlling output
model = model.pdb Output
model file name
chain_to_alignment_matching Chain-to-alignment matching options
consecutivity = *geometry
numbering Consecutivity criterion to detect chain breaks
min_hss_length = 3 Minimum length of a sequence
fragment to be included in chain alignment
max_seed_hss_count = 12 Number of HSS to use in extensive
search
max_completion_hss_count = 6 Number of HSS to use in gap filling
min_sequence_overlap = 10
Minimum overlap between sequences to perform full alignment
min_sequence_identity = 0.80 Minimum
sequence identity of accepted chain alignment
homology_modellingParameters controlling homology
modelling
min_residue_margin = 2 Minimum number of residues to cut back on both sides of
loops
residue_distance = 2.5 Distance covered by a single residue (in Å)
min_edge_segment_length = 5

Discard edge segments if shorter (after considering gap margins)min_internal_segment_length = 2 Discard internal segments if shorter (after considering gap margins)max_loop_length = 60 Maximum loop lengthrosetta_max_build_attempts = 1000 Maximum build attempts to close looprosetta_bump_overlap_factor = 0.1 Allows some atomic overlap in initial loop closuresrosetta_loop_closure = kic *ngk Algorithm for Rosetta loop closurerosetta_loop_refinement = *no default quick fast test Algorithm for Rosetta loop refinement

- inputParameters controlling inputstructure = None PDB file namealignment = None Alignment file namesequence = None Target sequence
- structure = None PDB file name
- alignment = None Alignment file name
- sequence = None Target sequence
- outputParameters controlling outputmodel = model.pdb Output model file name
- model = model.pdb Output model file name
- chain_to_alignment_matchingChain-to-alignment matching optionsconsecutivity = *geometry numbering Consecutivity criterion to detect chain breaksmin_hss_length = 3 Minimum length of a sequence fragment to be included in chain alignmentmax_seed_hss_count = 12 Number of HSS to use in extensive searchmax_completion_hss_count = 6 Number of HSS to use in gap fillingmin_sequence_overlap = 10 Minimum overlap between sequences to perform full alignmentmin_sequence_identity = 0.80 Minimum sequence identity of accepted chain alignment
- consecutivity = *geometry numbering Consecutivity criterion to detect chain breaks
- min_hss_length = 3 Minimum length of a sequence fragment to be included in chain alignment
- max_seed_hss_count = 12 Number of HSS to use in extensive search
- max_completion_hss_count = 6 Number of HSS to use in gap filling
- min_sequence_overlap = 10 Minimum overlap between sequences to perform full alignment
- min_sequence_identity = 0.80 Minimum sequence identity of accepted chain alignment
- homology_modellingParameters controlling homology modellingmin_residue_margin = 2 Minimum number of residues to cut back on both sides of loopsresidue_distance = 2.5 Distance covered by a single residue (in A)min_edge_segment_length = 5 Discard edge segments if shorter (after considering gap margins)min_internal_segment_length = 2 Discard internal segments if shorter (after considering gap margins)max_loop_length = 60 Maximum loop lengthrosetta_max_build_attempts = 1000 Maximum build attempts to close looprosetta_bump_overlap_factor = 0.1 Allows some atomic overlap in initial loop closuresrosetta_loop_closure = kic *ngk Algorithm for Rosetta loop closurerosetta_loop_refinement = *no default quick fast test Algorithm for Rosetta loop refinement
- min_residue_margin = 2 Minimum number of residues to cut back on both sides of loops
- residue_distance = 2.5 Distance covered by a single residue (in A)
- min_edge_segment_length = 5 Discard edge segments if shorter (after considering gap margins)
- min_internal_segment_length = 2 Discard internal segments if shorter (after considering gap margins)
- max_loop_length = 60 Maximum loop length
- rosetta_max_build_attempts = 1000 Maximum build attempts to close loop
- rosetta_bump_overlap_factor = 0.1 Allows some atomic overlap in initial loop closures
- rosetta_loop_closure = kic *ngk Algorithm for Rosetta loop closure

- rosetta_loop_refinement = *no default quick fast test Algorithm for Rosetta loop refinement

Finding NCS in chains from a PDB file with `simple_ncs_from_pdb`

`simple_ncs_from_pdb.html`

Contents

- Author(s)
- Purpose
- Usage for searching How `simple_ncs_from_pdb` works: Additional notes on how `simple_ncs_from_pdb` works: Examples Other outputs Possible Problems Specific limitations and problems:
- How `simple_ncs_from_pdb` works:
- Additional notes on how `simple_ncs_from_pdb` works:
- Examples
- Other outputs
- Possible Problems
- Specific limitations and problems:
- Usage for validation Why we filter annotations How we filter annotations What you may see in the log file (example) General recommendations when you encounter this situation in refinement
- Why we filter annotations
- How we filter annotations
- What you may see in the log file (example)
- General recommendations when you encounter this situation in refinement
- List of all available keywords

Author(s)

- `simple_ncs_from_pdb` : Tom Terwilliger, Youval Dar, Oleg Sobolev
- Phil command interpreter: Ralf W. Grosse-Kunstleve
- `find_domain`: Peter Zwart

Purpose

- The `simple_ncs_from_pdb` method identifies NCS in the chains in a PDB file and writes out the NCS operators in forms suitable for phenix.refine, resolve, and the AutoSol and AutoBuild Wizards.
- Validate NCS annotations against a model

Usage for searching

How `simple_ncs_from_pdb` works:

The basic steps that the `simple_ncs_from_pdb` carries out are:

- Remove part of the structure specified in `exclude_selection` parameter.

- Identify sets of matching segments in chains in the PDB file by sequences (use value of `chain_similarity_threshold` as cutoff). These are potential NCS-related chains.
- Determine which matching segments have overall rms distance (RMSD) within the given tolerance (`chain_max_rmsd`, typically 2 Å)
- Remove residues from matching segments if locally spatially misaligned (`residue_match_radius`)
- Remove atoms from matching residues if not in both residues (for example, if a side chain is missing in one of the matching residues)
- Check atoms order residues

Additional notes on how `simple_ncs_from_pdb` works:

The chains matching is done using dynamic programming alignment of residues and atoms. The first pass contains restriction on minimal similarity, set by `min_percent` (number of matching residues)/(number of residues in longer chain)

From the matching chains list, we remove chain pairs where the matching segments exceed the RMSD limit.

Matching segments are scanned for local residue misalignment. Residues where $(\text{max_atom_distance} - \text{min_atom_distance}) > \text{match_radius}$ are excluded from matching segment. This allow local differences in matching chains.

If matching residues have different number of atoms (For example if one containing the side chain while the other not), only the matching atoms will be included.

Grouping of chains is not performed.

Alternative conformation are excluded from matching segments.

The matching is done by the residues name strings, not by the residue numbers, this allows handling of insertions in PDB file.

The result of the NCS search is combination of NCS related groups and invariant or non-NCS related regions, to the atom level. In every NCS group all copies have the same number of atoms and can be reproduced by applying the appropriate rotation and translation to the master copy.

Examples

When running

```
phenix.simple_ncs_from_pdb 2h50.pdb
```

The following files will be produced

- NCS operators written in format for `phenix.refine`
`2h50_simple_ncs_from_pdb.phil`
- NCS operators written in format for the PHENIX Wizards (if `write_spec_files=True`)

```
2h50_simple_ncs_from_pdb.ncs_spec
2h50_simple_ncs_from_pdb.resolve
```

The file that should be used for refinement is `4boz_simple_ncs_from_pdb.phil`. This file can also be modified if a particular NCS relations need to be changed. The content of that file is the exact selection sting of the atoms in the NCS groups

The content of `2h50_simple_ncs_from_pdb.ncs` is

```

ncs_group {
  reference = chain 'A'
  selection = chain 'C'
  selection = chain 'E'
  selection = chain 'G'
  selection = chain 'I'
  selection = chain 'K'
  selection = chain 'M'
  selection = chain 'O'
  selection = chain 'Q'
  selection = chain 'S'
  selection = chain 'U'
  selection = chain 'W'
}

```

Other outputs

To get output in other format:

```
phenix.simple_ncs_from_pdb 4boz.pdb write_spec_files=True
```

Simple_ncs_from_pdb will analyze the chains in 4boz.pdb and identify any NCS that exists. For this sample run the following output is produced:

```

GROUP 1
Summary of NCS group with 2 operators:
ID of chain/residue where these apply: [['A', 'D'], [[147, 150], [152, 211],
[213, 275], [280, 305], [307, 308]], [[147, 150], [152, 211], [213, 275], [280, 305], [307, 308]]]
RMSD (A) from chain A: 0.0 0.72
Number of residues matching chain A:[155, 155]

OPERATOR 1
CENTER: 24.4880 -13.3177 -20.1848

ROTA 1: 1.0000 0.0000 0.0000
ROTA 2: 0.0000 1.0000 0.0000
ROTA 3: 0.0000 0.0000 1.0000
TRANS: 0.0000 0.0000 0.0000

OPERATOR 2
CENTER: 15.9430 11.8822 0.6609

ROTA 1: 0.7964 -0.5651 0.2152
ROTA 2: -0.5503 -0.8249 -0.1295
ROTA 3: 0.2507 -0.0152 -0.9680
TRANS: 18.3631 5.3425 -23.3604

GROUP 2
Summary of NCS group with 3 operators:
ID of chain/residue where these apply: [['B', 'C', 'E'], [[1, 41], [43, 71]],
[[1, 41], [43, 71]], [[1, 41], [43, 71]]]
RMSD (A) from chain B: 0.0 0.8 0.77
Number of residues matching chain B:[70, 70, 70]

OPERATOR 1
CENTER: 47.5581 -9.5652 -26.8403

ROTA 1: 1.0000 0.0000 0.0000
ROTA 2: 0.0000 1.0000 0.0000
ROTA 3: 0.0000 0.0000 1.0000
TRANS: 0.0000 0.0000 0.0000

OPERATOR 2
CENTER: 27.7410 -11.0237 -49.7361

ROTA 1: -0.6661 0.2615 -0.6986

```

ROTA 2: 0.2866 -0.7749 -0.5634
ROTA 3: -0.6886 -0.5755 0.4412
TRANS: 34.1743 -54.0788 7.8610

OPERATOR 3
CENTER: 29.3812 -4.6379 12.2671

ROTA 1: 0.7539 -0.5901 0.2888
ROTA 2: -0.5860 -0.8027 -0.1106
ROTA 3: 0.2971 -0.0858 -0.9510
TRANS: 19.1286 5.2864 -24.3021

Another way to view the results is

phenix.simple_ncs_from_pdb 4boz.pdb show_summary=true

Chains in model:

A B C D E

NCS summary:

Number of NCS groups : 2

Group # : 1

Number of copies : 2

Chains in master : 'A'

Chains in copies : 'D'

Group # : 2

Number of copies : 3

Chains in master : 'B'

Chains in copies : 'C', 'E'

Transforms:

Group # : 1

Transform # : 1

RMSD : 0

ROTA 0 1.0000 0.0000 0.0000

ROTA 1 0.0000 1.0000 0.0000

ROTA 2 0.0000 0.0000 1.0000

TRANS 0.0000 0.0000 0.0000

Transform # : 2

RMSD : 0.720792956817

ROTA 0 0.7964 -0.5503 0.2507

ROTA 1 -0.5651 -0.8249 -0.0152

ROTA 2 0.2152 -0.1295 -0.9680

TRANS -5.8295 14.4282 -25.8708

Group # : 2

Transform # : 1

RMSD : 0

ROTA 0 1.0000 0.0000 0.0000

ROTA 1 0.0000 1.0000 0.0000

ROTA 2 0.0000 0.0000 1.0000

TRANS 0.0000 0.0000 0.0000

Transform # : 2

RMSD : 0.796197645664

ROTA 0 -0.6661 0.2866 -0.6886

ROTA 1 0.2615 -0.7749 -0.5755

ROTA 2 -0.6986 -0.5634 0.4412

TRANS 43.6766 -46.3175 -10.0616

Transform # : 3

RMSD : 0.772963874518

```

ROTA 0 0.7539 -0.5860 0.2971
ROTA 1 -0.5901 -0.8027 -0.0858
ROTA 2 0.2888 -0.1106 -0.9510
TRANS -4.1022 13.4462 -28.0502

```

There are 5 chains in the PDB file (A,B,C,D,E). In the first group the master is A the the copy D and in the second group the master is B and the copy is C and E. Chain C is not in any group.

```
RMSD (A) from chain A: 0.0 0.72
```

shows the RMSD of matching atoms between the master and every other copy, The list of numbers is the list of matching residues by residue number. Note that this is not the exact selection, as it appears in: 4boz_simple_ncs_from_pdb.ncs.

ROTA and TRANS are the rotation and translation information. In the .ncs_spec file and the default on-screen representation of the results, the rotation and translation are:

```
Master = ROT x Copy + TRANS
```

While in the CCBTX implementation the common use is:

```
Copy = ROT x Master + TRANS
```

So the rotation/Translation are the inverse of each other in the two formats. Both formats are displayed in the summary.

A portion of the contents of the 4boz_simple_ncs_from_pdb.ncs_spec file, which you can edit if you want and which you can use in the AutoBuild Wizard, are shown below. NOTE: The ncs operators describe how to map the N'th ncs-related copy on to the first copy.

```

Summary of NCS information
Thu Apr 2 15:44:03 2015
/net/cci-filer2/raid1/home/...

new_ncs_group
new_operator

rota_matrix 1.0000 0.0000 0.0000
rota_matrix 0.0000 1.0000 0.0000
rota_matrix 0.0000 0.0000 1.0000
tran_orth 0.0000 0.0000 0.0000

center_orth 24.4880 -13.3177 -20.1848
CHAIN A
RMSD 0
MATCHING 155
RESSEQ 147:150
RESSEQ 152:211
RESSEQ 213:275
RESSEQ 280:305
RESSEQ 307:308

new_operator

rota_matrix 0.7955 -0.5660 0.2164
rota_matrix -0.5511 -0.8242 -0.1300
rota_matrix 0.2520 -0.0159 -0.9676
tran_orth 18.4021 5.3569 -23.3621

center_orth 15.9430 11.8822 0.6609
CHAIN D
RMSD 0.817
MATCHING 155
RESSEQ 147:150
RESSEQ 152:211

```


RESSEQ 213:275
RESSEQ 280:305
RESSEQ 307:308

Possible Problems

- Simple_ncs_from_pdb cannot find NCS groups: try relaxing search parameters, e.g. set bigger chain_max_rmsd, smaller chain_similarity_threshold.

Simple_ncs_from_pdb cannot find NCS groups: try relaxing search parameters,

e.g. set bigger chain_max_rmsd, smaller chain_similarity_threshold.

e.g. set bigger chain_max_rmsd, smaller chain_similarity_threshold.

- Simple_ncs_from_pdb produces very complicated selections, like reference = (chain 'A' and (resid 147:150 or resid 152:194 or (resid 195 and (name N or name CA or name C or name O or name CB)) or resid 196:211 or resid 213:229 or (resid 230 and (name N or name CA or name C or name O or name CB or name CG)) or resid 231:298 or (resid 299 and (name N or name CA or name C or name O or name CB or name CG)) or resid 300:305 or resid 307:308)) This indicates that procedure is trying to exclude some residues or atoms from the selections. This could be caused by alternative conformations of residues (not supported), poor alignment of particular residues (try to increase residue_match_radius) or missing atoms in one of the chains.

Simple_ncs_from_pdb produces very complicated selections, like

```
reference = (chain 'A' and (resid 147:150 or resid 152:194 or (resid 195 and
(name N or name CA or name C or name O or name CB )) or resid 196:211 or
resid 213:229 or (resid 230 and (name N or name CA or name C or name O or name
CB or name CG )) or resid 231:298 or (resid 299 and (name N or name CA or name
C or name O or name CB or name CG )) or resid 300:305 or resid 307:308))
```

This indicates that procedure is trying to exclude some residues or atoms from the selections. This could be caused by alternative conformations of residues (not supported), poor alignment of particular residues (try to increase residue_match_radius) or missing atoms in one of the chains.

Specific limitations and problems:

- a large chain_similarity_threshold can prevent matching segments in chains that are of very different size.
- The value of chain_max_rmsd, the RMSD limit between chains should be set with care. Small value might exclude chains with problems in the model, even though they are NCS related. Large value might cause non-NCS related chains and to be constraint as related.
- The .ncs_spec files do not support representation of groups with multiple chains in master and copies. a group [(A,C),(B,D)] will be represented as [(A),(B)] and [(C),(D)], duplicating the rotation and translation operations.
- The .ncs_spec files do not support representation atoms, residue name and insertion selection, it only shows the numbers of the matching residues.
- The .ncs_spec files rotation and translation operation are

Master = ROT x Copy + TRANS

and not:

Copy = ROT x Master + TRANS

They are the inverse of the rotation and translation that are used in the implementation of the NCS relation.

Usage for validation

To run `simple_ncs_from_pdb` in validation mode one should supply NCS groups in form of phil file. The purpose of this mode is to illustrate how your NCS annotations will be filtered in `phenix.refine` and `phenix.real_space_refine`.

Why we filter annotations

We filter annotations because we need to be sure that:

- number of atoms in reference and selections are the same
- residues in reference and selections match each other
- matched residues have the same atoms in all copies

Without all these NCS restraints and constraints cannot work reliably. We understand that it could be tricky to manually create such selections, especially for poor models where some residues may be not modelled yet or lacking some atoms. Therefore we filter user-supplied annotations before refinement by default and there is no way to turn it off.

How we filter annotations

For every NCS group supplied by a user we:

- Select part of the model containing reference atoms and all selection atoms.

Select part of the model containing reference atoms and all selection atoms.

- Then we search for NCS in this sub-model using parameters from `ncs_search` scope with some modifications. Precisely: `chain_max_rmsd` or 10, whichever is greater `residue_match_radius` or 1000, whichever is greater `chain_similarity_threshold` or 0.5 whichever is smaller. These modifications are used to relax criteria for validation to a large extent to accomodate poor matches in poor models.

Then we search for NCS in this sub-model using parameters from `ncs_search` scope with some modifications. Precisely:

- `chain_max_rmsd` or 10, whichever is greater
- `residue_match_radius` or 1000, whichever is greater
- `chain_similarity_threshold` or 0.5 whichever is smaller.

These modifications are used to relax criteria for validation to a large extent to accomodate poor matches in poor models.

- If found NCS relations cover all atoms selected in 1, validation passed, the NCS group is used as was defined by a user. If found NCS relations do not cover all the initial atoms selected in 1, we create new NCS selections and use them further in refinement. If no NCS relations were found, the whole NCS group is discarded.

If found NCS relations cover all atoms selected in 1, validation passed, the NCS group is used as was defined by a user. If found NCS relations do not cover all the initial atoms selected in 1, we create new NCS selections and use them further in refinement. If no NCS relations were found, the whole NCS group is discarded.

Note, we only filter supplied selections. Nothing extra will be added.

What you may see in the log file (example)

One may find something like this in the log-file:

```
Validating user-supplied NCS groups...
Validating:
ncs_group {
  reference = "(chain A and resid 11:376 and not (resid 19:31 or resid 42:67 or resid 70:123 or resid 132:145 or
  selection = "(chain C and resid 11:376 and not (resid 19:31 or resid 42:67 or resid 70:123 or resid 132:145 or
}
MODIFIED. Some of the atoms were excluded from your selection.
The most common reasons are:
1. Missing residues in one or several copies in NCS group.
2. Presence of alternative conformations (they are excluded).
3. Residue mismatch in requested copies.
Please check the validated selection further down.
```

and further down something like:

```
Found NCS groups:
ncs_group {
  reference = (chain 'A' and (resid 11 through 18 or resid 32 through 41 or resid 68 through 6 \
9 or resid 124 through 125 or resid 129 through 131 or resid 146 through 147 or \
resid 185 through 189 or resid 250 through 338 or resid 358 through 364 or resid \
375 through 376))
  selection = (chain 'C' and (resid 11 through 18 or resid 32 through 41 or resid 68 through 6 \
9 or resid 124 through 131 or resid 146 through 147 or resid 185 through 189 or \
resid 250 through 338 or resid 358 through 376))
}
```

This is how NCS selections were modified for one of the reasons listed above. In this example, they are essentially the same with slightly different syntax. Let's look at the differences.

- In chain A residues 126-128 are excluded from selection. Indeed, there were no such residues in chain C.
- In chain A residues 365-374 are excluded. Same as #1, absent in chain C.

These two are the only differences from your selection and they don't change anything! They look much different because we have to construct new selections from scratch (internal representation of selections) automatically.

When one sees something like this in new selections:

```
(resid 28 and (name N or name CA or name C or name O or name CB or name CG or name CD ))
```

this is usually because another chain is lacking particular atom(s). Therefore we select particular atoms of the residue that present in all other copies. In this particular case, residue 28 in chain A has NE and CZ atoms, while chain B lacks them.

Other messages one can see is:

```
REJECTED because copies don't match good enough.
Try to revise selections or adjust chain_similarity_threshold or
chain_max_rmsd parameters.

OK. All atoms were included in
validated selection.
```

General recommendations when you encounter this situation in refinement

- If resulting selections (Found NCS groups) looks good to you - you should be fine, NCS is being applied and you can see exactly what selections are being used.
- Check the filtered selections. Maybe you will want to modify the model (put on missing atoms or remove extras).

- If you want your selections to be preserved - make them such that they satisfy all requirements in Why we filter annotations.

List of all available keywords

pdb_in = None Input PDB file to be used to identify ncstemp_dir = "" temporary directory (ncs_domain_pdb will be written there)ncs_domain_pdb_stem = None NCS domains will be written to ncs_domain_pdb_stem+"group_"+nnwrite_ncs_domain_pdb = False You can write out PDB files representing NCS domains for density modification if you wantwrite_ncs_phil = False Write out phil parameters for the NCS groups to file in working directoryshow_summary = False Show NCS information summarywrite_spec_files = False Write .ncs_spec and .resolve representation of NCS groupsncs_searchSet of parameters for NCS search procedure. Some of them also used for filtering user-supplied ncs_group.enabled = False Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).exclude_selection = "element H or element D or water" Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in NCS.chain_similarity_threshold = 0.85 Threshold for sequence similarity between matching chains. A smaller value may cause more chains to be grouped together and can lower the number of common residueschain_max_rmsd = 2. limit of rms difference between chains to be considered as copiesresidue_match_radius = 4.0 Maximum allowed distance difference between pairs of matching atoms of two residuesry_shortcuts = False Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.minimum_number_of_atoms_in_copy = 3 Do not create ncs groups where master and copies would contain less than specified amount of atomsvalidate_user_supplied_groups = True Enable validation of user-supplied ncs_group. Need to exercise a lot of caution turning this off. This option is for developers only.ncs_groupThe definition of one NCS group. Note, that almost always in refinement programs they will be checked and filtered if needed.reference = None Residue selection string for the complete master NCS copyselection = None Residue selection string for each NCS copy location in ASU

- pdb_in = None Input PDB file to be used to identify ncs
- temp_dir = "" temporary directory (ncs_domain_pdb will be written there)
- ncs_domain_pdb_stem = None NCS domains will be written to ncs_domain_pdb_stem+"group_"+nn
- write_ncs_domain_pdb = False You can write out PDB files representing NCS domains for density modification if you want
- write_ncs_phil = False Write out phil parameters for the NCS groups to file in working directory
- show_summary = False Show NCS information summary
- write_spec_files = False Write .ncs_spec and .resolve representation of NCS groups
- ncs_searchSet of parameters for NCS search procedure. Some of them also used for filtering user-supplied ncs_group.enabled = False Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).exclude_selection = "element H or element D or water" Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in NCS.chain_similarity_threshold = 0.85 Threshold for sequence similarity between matching chains. A smaller value may cause more chains to be grouped together and can lower the number of common residueschain_max_rmsd = 2. limit of rms difference between chains to be considered as copiesresidue_match_radius = 4.0 Maximum allowed distance difference between pairs of matching atoms of two residuesry_shortcuts = False Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.minimum_number_of_atoms_in_copy =

3 Do not create ncs groups where master and copies would contain less than specified amount of atoms
validate_user_supplied_groups = True Enable validation of user-supplied ncs_group. Need to exercise a lot of caution turning this off. This option is for developers only.

- enabled = False Enable NCS restraints or constraints in refinement (in some cases may be switched on inside refinement program).
- exclude_selection = "element H or element D or water" Atoms selected by this selection will be excluded from the model before any NCS search and/or filtering procedures. There is no way atoms defined by this selection will be in NCS.
- chain_similarity_threshold = 0.85 Threshold for sequence similarity between matching chains. A smaller value may cause more chains to be grouped together and can lower the number of common residues
- chain_max_rmsd = 2. limit of rms difference between chains to be considered as copies
- residue_match_radius = 4.0 Maximum allowed distance difference between pairs of matching atoms of two residues
- try_shortcuts = False Try very quick check to speed up the search when chains are identical. If failed, regular search will be performed automatically.
- minimum_number_of_atoms_in_copy = 3 Do not create ncs groups where master and copies would contain less than specified amount of atoms
- validate_user_supplied_groups = True Enable validation of user-supplied ncs_group. Need to exercise a lot of caution turning this off. This option is for developers only.
- ncs_group The definition of one NCS group. Note, that almost always in refinement programs they will be checked and filtered if needed.
reference = None Residue selection string for the complete master
NCS copy
selection = None Residue selection string for each NCS copy location in ASU
- reference = None Residue selection string for the complete master NCS copy
- selection = None Residue selection string for each NCS copy location in ASU

Sorting heteroatoms

sort_hetatms.html

The program `phenix.sort_hetatms` is a utility designed to re-group the non-polymeric molecules (heteroatoms) in a model in a roughly similar manner to the format used by the Protein Data Bank. This consists of matching all heteroatoms to the nearest polymer chain, resetting the chain IDs and residue numbers, converting ATOM labels to HETATM, and optionally sorting waters by B-factor. It is primarily intended for internal use, and is also run within `phenix.ligand_pipeline`. The only input required is a PDB file, since the program does not deal with molecular geometry or experimental data. Any REMARK records will be removed, but this can be disabled by setting `preserve_remarks=True`.

Note that the PDB will, inevitably, further modify the contents of the model according to its own rules. However, the program output is significantly closer to the PDB conventions than the output of `phenix.refine` or similar programs.

List of all available parameters

`file_name = None` Input file
`output_file = None` Output file
`unit_cell = None` Space group
`space_group = None` Ignore symmetry = False Don't take symmetry-related chains into account when determining the nearest macromolecule.
`preserve_remarks = False` Propagate all REMARK records to the output file.
`verbose = False`
`remove_hetatm_ter_records = True` The official PDB format only allows TER records at the end of polymer chains, whereas the CCTBX PDB-handling tools will insert TER after each chain of any type. If this parameter is True, the extra TER records will be removed.
`preserve_chain_id = False` The default behavior is to group heteroatoms with the nearest macromolecule chain, whose ID is inherited. This parameter disables the change of chain ID, and preserves the original chain ID.
`waters_only = False` Rearrange waters, but leave all other ligands alone.
`sort_waters_by = None` *b_iso Ordering of waters - by default it will sort them by the isotropic B-factor.
`set_hetatm_record = True` Convert ATOM to HETATM where appropriate.
`ignore_selection = None` Selection of atoms to skip. Any residue group which overlaps with this selection will be preserved with the original chain ID and numbering.
`renumber = True` Renumber heteroatoms once they are in new chains.
`sequential_numbering = True` If True, the heteroatoms will be renumbered starting from the next available residue number after the end of the associated macromolecule chain. Otherwise, numbering will start from 1.
`distance_cutoff = 6.0` Cutoff for identifying nearby macromolecule chains. This should be kept relatively small for speed reasons, but it may miss waters that are far out in solvent channels.
`remove_waters_outside_radius = False` Remove waters more than the specified distance cutoff to the nearest polymer chain (to avoid PDB complaints).
`loose_chain_id = X` Chain ID assigned to heteroatoms that can't be mapped to a nearby macromolecule chain.
`model_skip_expand_with_mtrix = False` If set to false then expand the model into NCS copies using matrix (MTRIX) records present in PDB file.
`job_title = None` Job title in PHENIX GUI, not used on command line

- `file_name = None` Input file
- `output_file = None`
- `unit_cell = None`
- `space_group = None`
- `ignore_symmetry = False` Don't take symmetry-related chains into account when determining the nearest macromolecule.
- `preserve_remarks = False` Propagate all REMARK records to the output file.

- `verbose = False`
- `remove_hetatm_ter_records = True` The official PDB format only allows TER records at the end of polymer chains, whereas the CCTBX PDB-handling tools will insert TER after each chain of any type. If this parameter is True, the extra TER records will be removed.
- `preserve_chain_id = False` The default behavior is to group heteroatoms with the nearest macromolecule chain, whose ID is inherited. This parameter disables the change of chain ID, and preserves the original chain ID.
- `waters_only = False` Rearrange waters, but leave all other ligands alone.
- `sort_waters_by = None` *`b_iso` Ordering of waters - by default it will sort them by the isotropic B-factor.
- `set_hetatm_record = True` Convert ATOM to HETATM where appropriate.
- `ignore_selection = None` Selection of atoms to skip. Any residue group which overlaps with this selection will be preserved with the original chain ID and numbering.
- `renumber = True` Renumber heteroatoms once they are in new chains.
- `sequential_numbering = True` If True, the heteroatoms will be renumbered starting from the next available residue number after the end of the associated macromolecule chain. Otherwise, numbering will start from 1.
- `distance_cutoff = 6.0` Cutoff for identifying nearby macromolecule chains. This should be kept relatively small for speed reasons, but it may miss waters that are far out in solvent channels.
- `remove_waters_outside_radius = False` Remove waters more than the specified distance cutoff to the nearest polymer chain (to avoid PDB complaints).
- `loose_chain_id = X` Chain ID assigned to heteroatoms that can't be mapped to a nearby macromolecule chain.
- `model_skip_expand_with_mtrix = False` If set to false then expand the model into NCS copies using matrix (MTRIX) records present in PDB file
- `job_title = None` Job title in PHENIX GUI, not used on command line

phenix.structural_domain_search

structural_domain_search.html

Contents

- Algorithm
- Usage Important parameters Output
- Important parameters
- Output
- References
- Command-line options

Finds domains by calculating low cuts in the interaction graph, and evaluates whether the cut would correspond to a viable domain decomposition.

Algorithm

Assuming that interaction within a domain are much stronger than between domains, a domain can be considered a low cut within a suitable constructed interaction graph.

The structure is divided up into chunks, and an interaction network is calculated. This is then turned into an interaction graph, and low cuts are enumerated. Cuts are then enumerated whether they would correspond to viable domains, based on size, compactness and interface strength, and viable domains are stored. The procedure is then repeated recursively until no viable cuts can be found.

Usage

The only required input to the program is a structure file (PDB/cif), which will then be analysed. The domain finding algorithm is then repeated for each protein chains, and results will be written to the logfile, and also into PHIL-syntax .eff-files.

The simplest command line:

```
phenix.structural_domain_search example.pdb
```

Runtime depends on the structure size, typically 1 min for structures between 200-300 residues, and 10 min for structures up to 1000 residues.

Important parameters

- `max_iterations`: controls how many low cuts to enumerate. Increasing this number will make the search more exhaustive, but also slower.
- `sufficient_count`: terminate the search early if the requested number of cuts have been found. It is probably a good idea to search for more than one, since the lowest cut does not always correspond to the highest domain quality score, but the default value (30) is perhaps slightly conservative, and good results can be achieved with `sufficient_count` = 10. On the other hand, the underlying clustering algorithm gets very slow when more than 100 values need to be clustered, and hence this is a practical upper limit.
- `min_size`: minimum size for an acceptable domain. Domains smaller than this will not be accepted. Default is 30.

- `prune_fraction`: controls whether to simplify the interaction graph by joining low-connected vertices. This can make the correct domain decomposition to appear earlier and hence make the search more efficient, at a slight loss of cut precision. A typical `prune_fraction` is 0.05-0.15.
- `algorithm`: the quick algorithm typically gives correct results, but for some tricky (especially multiple domain) structures the thorough algorithm may find a better decomposition. Running times up to 10 times of the quick algorithm are not uncommon.

Output

The program writes out the best domain decomposition for each chain, as a series of .eff-files containing the residue segments.

All considered decompositions are recorded in the logfile.

References

Efficient Algorithms for the Problems of Enumerating Cut by Non-decreasing Weights. L.-P. Yeh, B.-F. Wang, and H.-H. Su. *Algorithmica* 5626, 297-312 (2010).

- Efficient Algorithms for the Problems of Enumerating Cut by Non-decreasing Weights. L.-P. Yeh, B.-F. Wang, and H.-H. Su. *Algorithmica* 5626, 297-312 (2010).

A simple min-cut algorithm. M. Stoer, and F. Wagner. *Journal of the ACM* 44, 585-591 (1997).

- A simple min-cut algorithm. M. Stoer, and F. Wagner. *Journal of the ACM* 44, 585-591 (1997).

An Experimental Comparison of Min-Cut/Max-Flow Algorithms for Energy Minimization in Vision. Y. Boykov, and V. Kolmogorov. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26, 1124-1137 (2004).

- An Experimental Comparison of Min-Cut/Max-Flow Algorithms for Energy Minimization in Vision. Y. Boykov, and V. Kolmogorov. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26, 1124-1137 (2004).

Protein domain decomposition using a graph-theoretic approach. Y. Xu, D. Xu, and H.N. Gabow. *Bioinformatics* 16, 1091-1104 (2000).

- Protein domain decomposition using a graph-theoretic approach. Y. Xu, D. Xu, and H.N. Gabow. *Bioinformatics* 16, 1091-1104 (2000).

Command-line options

inputInput filesstructure = None Structure fileoutputOutput filesroot = "domain" Root for domain
 fileswrite_pdb_file = False Write PDB file for each domainconfigurationCalculation
 parametersinteraction_strengthParameters to calculate interaction strength between two
 residuessignificance_threshold = 0.01 Discard interaction if below this levelprune_fraction = 0.00 Discard
 lowest connected segmentslong_range_contactParameters for defining a long-range
 contactmax_spatial_distance = 4.0 Maximum distance for atom-atom contactshort_range_contactParameters
 controlling the strength of short range contactsdefault = 10.0 Strength of a coil-to-coil and mixed type
 connectionshelix_to_helix_connection = 50.0 Strength of a helix-to-helix
 connectionstrand_to_strand_connection = 50.0 Strength of a strand-to-strand
 connectionassembly_membershipParameters rewarding membership in a higher order assemblysheet_bonus
 = 20.0 Extra strength if two strands are within the same sheetchunkingParameters controlling chunking of
 segmentscoil_max_size = 15 Minimum chunksize for a coilhelix_max_size = None Minimum chunksize for a
 helixstrand_max_size = None Minimum chunksize for a strandevaluationParameters controlling domain

acceptancemin_size = 30 Minimum domain size is residuesmin_compactness = 0.6 Minimum compactness (internal interaction strength to atoms ratio)max_interface_strength = 0.4 Maximum interface strength to internal interaction strength ratiosearchParameters controlling domain searchalgorithm = *quick thorough Algorithm to use
 max_macrocycles = 5 Maximum number of macrocyclesmax_iterations = 500 Maximum cut iterations to perform
 sufficient_count = 31 Quit search if sufficient number of domains has been locatedcluster_radius = 20 Radius for identifying quasi-equivalent domains
 max_residue_difference_for_equality = 10 Maximum residue differences between two domains
 max_residue_overlap_for_compatibility = 5 Max number of common residues between two domains
 strong_connection = 100 Do not separate multisegment domains that are at least as strongly connected

- inputInput filesstructure = None Structure file
- structure = None Structure file
- outputOutput filesroot = "domain" Root for domain fileswrite_pdb_file = False Write PDB file for each domain
- root = "domain" Root for domain files
- write_pdb_file = False Write PDB file for each domain
- configurationCalculation parametersinteraction_strengthParameters to calculate interaction strength between two residues
 significance_threshold = 0.01 Discard interaction if below this level
 prune_fraction = 0.00 Discard lowest connected segments
 long_range_contactParameters for defining a long-range contact
 max_spatial_distance = 4.0 Maximum distance for atom-atom
 contactshort_range_contactParameters controlling the strength of short range contacts
 default = 10.0 Strength of a coil-to-coil and mixed type connections
 helix_to_helix_connection = 50.0 Strength of a helix-to-helix connection
 strand_to_strand_connection = 50.0 Strength of a strand-to-strand connection
 assembly_membershipParameters rewarding membership in a higher order assembly
 sheet_bonus = 20.0 Extra strength if two strands are within the same sheet
 chunkingParameters controlling chunking of segments
 coil_max_size = 15 Minimum chunksize for a coil
 helix_max_size = None Minimum chunksize for a helix
 strand_max_size = None Minimum chunksize for a strand
 evaluationParameters controlling domain
 acceptancemin_size = 30 Minimum domain size is residuesmin_compactness = 0.6 Minimum compactness (internal interaction strength to atoms ratio)max_interface_strength = 0.4 Maximum interface strength to internal interaction strength ratio
 searchParameters controlling domain searchalgorithm = *quick thorough Algorithm to use
 max_macrocycles = 5 Maximum number of macrocyclesmax_iterations = 500 Maximum cut iterations to perform
 sufficient_count = 31 Quit search if sufficient number of domains has been located
 cluster_radius = 20 Radius for identifying quasi-equivalent domains
 max_residue_difference_for_equality = 10 Maximum residue differences between two domains
 max_residue_overlap_for_compatibility = 5 Max number of common residues between two domains
 strong_connection = 100 Do not separate multisegment domains that are at least as strongly connected
- interaction_strengthParameters to calculate interaction strength between two residues
 significance_threshold = 0.01 Discard interaction if below this level
 prune_fraction = 0.00 Discard lowest connected segments
 long_range_contactParameters for defining a long-range contact
 max_spatial_distance = 4.0 Maximum distance for atom-atom
 contactshort_range_contactParameters controlling the strength of short range contacts
 default = 10.0 Strength of a coil-to-coil and mixed type connections
 helix_to_helix_connection = 50.0 Strength of a helix-to-helix connection
 strand_to_strand_connection = 50.0 Strength of a strand-to-strand connection
 assembly_membershipParameters rewarding membership in a higher order

assemblysheet_bonus = 20.0 Extra strength if two strands are within the same sheet

- significance_threshold = 0.01 Discard interaction if below this level
- prune_fraction = 0.00 Discard lowest connected segments
- long_range_contactParameters for defining a long-range contactmax_spatial_distance = 4.0 Maximum distance for atom-atom contact
- max_spatial_distance = 4.0 Maximum distance for atom-atom contact
- short_range_contactParameters controlling the strength of short range contactsdefault = 10.0 Strength of a coil-to-coil and mixed type connectionshelix_to_helix_connection = 50.0 Strength of a helix-to-helix connectionstrand_to_strand_connection = 50.0 Strength of a strand-to-strand connection
- default = 10.0 Strength of a coil-to-coil and mixed type connections
- helix_to_helix_connection = 50.0 Strength of a helix-to-helix connection
- strand_to_strand_connection = 50.0 Strength of a strand-to-strand connection
- assembly_membershipParameters rewarding membership in a higher order assemblysheet_bonus = 20.0 Extra strength if two strands are within the same sheet
- sheet_bonus = 20.0 Extra strength if two strands are within the same sheet
- chunkingParameters controlling chunking of segmentscoil_max_size = 15 Minimum chunksize for a coilhelix_max_size = None Minimum chunksize for a helixstrand_max_size = None Minimum chunksize for a strand
- coil_max_size = 15 Minimum chunksize for a coil
- helix_max_size = None Minimum chunksize for a helix
- strand_max_size = None Minimum chunksize for a strand
- evaluationParameters controlling domain acceptancemin_size = 30 Minimum domain size is residuesmin_compactness = 0.6 Minimum compactness (internal interaction strength to atoms ratio)max_interface_strength = 0.4 Maximum interface strength to internal interaction strength ratio
- min_size = 30 Minimum domain size is residues
- min_compactness = 0.6 Minimum compactness (internal interaction strength to atoms ratio)
- max_interface_strength = 0.4 Maximum interface strength to internal interaction strength ratio
- searchParameters controlling domain searchalgorithm = *quick thorough Algorithm to useuse_max_macrocycles = 5 Maximum number of macrocyclesmax_iterations = 500 Maximum cut iterations to performssufficient_count = 31 Quit search if sufficient number of domains has been locatedcluster_radius = 20 Radius for identifying quasi-equivalent domainsmax_residue_difference_for_equality = 10 Maximum residue differences between two domainsmax_residue_overlap_for_compatibility = 5 Max number of common residues between two domainsstrong_connection = 100 Do not separate multisegment domains that are at least as strongly connected
- algorithm = *quick thorough Algorithm to use
- max_macrocycles = 5 Maximum number of macrocycles
- max_iterations = 500 Maximum cut iterations to perform
- sufficient_count = 31 Quit search if sufficient number of domains has been located
- cluster_radius = 20 Radius for identifying quasi-equivalent domains
- max_residue_difference_for_equality = 10 Maximum residue differences between two domains

- `max_residue_overlap_for_compatibility` = 5 Max number of common residues between two domains
- `strong_connection` = 100 Do not separate multisegment domains that are at least as strongly connected

Comparing related structures in the Phenix GUI

structure_comparison.html

Contents

- Authors
- Introduction
- Limitations
- Video Tutorial
- Configuration and required inputs
- Handling heterogeneous sequences
- Results
- Interacting with Coot
- Reference
- Citations

Authors

Programming: Nigel W. Moriarty, Dorothee Liebschner, Nat Echols, Billy K. Poon, Pavel Afonine, Jeff Headd (LBL) Concept: Herb Klei (BMS)

- Programming: Nigel W. Moriarty, Dorothee Liebschner, Nat Echols, Billy K. Poon, Pavel Afonine, Jeff Headd (LBL)
- Concept: Herb Klei (BMS)

Introduction

The structure comparison GUI is a tool for parallel validation and analysis of near-identical protein structures, such as different crystal forms, various mutants, or NCS-related copies. It displays validation outliers and regions of difference for a set of individual chains on a spreadsheet-like grid, which is linked to the graphics windows (Coot and PyMOL). If desired, both the extracted chains and electron density maps can be superimposed into a common frame of reference. These chains may subsequently be edited in Coot to ensure consistency and/or fix errors, and recovered in their original orientations for further refinement and rebuilding.

We have included a set of representative structures of human Protein Kinase A (PKA) for comparison in the Phenix distribution. To automatically set up a new project with these data, click "New Project" in the main GUI toolbar and then "Set up tutorial data..." in the dialog which appears, and select "Protein Kinase A (Structure Comparison)" from the drop-down menu.

Limitations

Only protein chains are supported at this time. RNA/DNA are not supported. Only X-ray diffraction data are supported. It is planned to extend usage for cryo-EM and neutron data in the future. Only a single set of chains can be compared at a time; if you have a set of heterogeneous complexes, the components need to be dealt with separately. This tool is of limited use for comparing proteins which diverge significantly in sequence; the program will quit with a warning if the fractional identity drops below 0.8 (this can be adjusted if desired).

See below for additional details.

- Only protein chains are supported at this time. RNA/DNA are not supported.
- Only X-ray diffraction data are supported. It is planned to extend usage for cryo-EM and neutron data in the future.
- Only a single set of chains can be compared at a time; if you have a set of heterogeneous complexes, the components need to be dealt with separately.
- This tool is of limited use for comparing proteins which diverge significantly in sequence; the program will quit with a warning if the fractional identity drops below 0.8 (this can be adjusted if desired). See below for additional details.

Video Tutorial

The tutorial video is available on the Phenix YouTube channel and covers the following topics:

- Basic overview
- How to run Structure Comparison
- Example of a Structure Comparison run using the PKA tutorial data

Configuration and required inputs

At a minimum, two or more PDB files containing near-identical chains are needed. A sequence file containing a single amino-acid sequence can be used to define the chain to be compared. It is recommended to also provide either X-ray data or pre-calculated map coefficients (the latter are preferred since these calculations are the slowest part of the program). The files generated by phenix.refine or phenix.maps are suitable for the latter. Each PDB file may contain an unlimited number of matching chains; in the PKA example, the structure 1syk has two copies of the catalytic subunit, while the rest contain only one. Non-matching chains will be ignored.

There are several options to open files or file pairs:

- Files may be dragged into the "Input files" window in the center of the Structure Comparison GUI.

Files may be dragged into the "Input files" window in the center of the Structure Comparison GUI.

- Files can be opened one by one by using the "Add File" button.

Files can be opened one by one by using the "Add File" button.

- If you have a directory containing multiple allowed file types, you may simply drag the directory in, and it will automatically guess the appropriate use for each file. If the PDB and reflection files use similar root filenames, or occur in pairs in the same directories, the association between them will be guessed automatically. For instance, the following pair of files will automatically be recognized as model and experimental data: 1syk.pdb 1syk.mtz while these will be recognized as model and map coefficients: 1syk.pdb 1syk_map_coeffs.mtz

If you have a directory containing multiple allowed file types, you may simply drag the directory in, and it will automatically guess the appropriate use for each file. If the PDB and reflection files use similar root filenames, or occur in pairs in the same directories, the association between them will be guessed automatically. For instance, the following pair of files will automatically be recognized as model and experimental data:

```
1syk.pdb
1syk.mtz
```

while these will be recognized as model and map coefficients:

```
1syk.pdb
1syk_map_coeffs.mtz
```

- Similarly, the "Add Directory" button can be used to open a directory and the program tries to automatically guess the file pairs.

Similarly, the "Add Directory" button can be used to open a directory and the program tries to automatically guess the file pairs.

- "Add Directory Tree" searches for file pairs in a directory tree.

"Add Directory Tree" searches for file pairs in a directory tree.

The map calculations have been parallelized, and this part of the program scales extremely well over multiple processors. The GUI will automatically be set to use `ncpu=1` processes, but you may increase this if desired.

The most important configurable option is the choice of superposing the models and maps prior for viewing together in Coot or PyMOL. This is very advantageous for comparing regions of difference, instead of switching between views for each chain. However, it has the drawback of removing the chains being analyzed from their original context, possibly obscuring other features that account for the observed structures. If you choose to turn on the superpositioning, the extracted chains will be written out as separate files in the results directory (along with reoriented maps for each chain, if available). Otherwise, the original models will be used for viewing. (Note that the analyses which depend on local environment in the crystal will still be run on the original models, and the coordinates mapped to their new positions.)

In addition to the models actually being compared, you may also provide a "reference model" on which all other chains will be superposed. If a reference model is not explicitly provided, the first chain found in the list of input files will be used instead. However, it may be advantageous to explicitly define this model if you are concerned about the quality or consistency of the structural alignments.

Handling heterogeneous sequences

Homologous structures and insertion codes are supported. To ensure that the residue numbering is interpreted correctly, you will need to uncheck the box labeled "Assume identical chain numbering". The sequence provided as input will be used to pick out similar chains, which will then be aligned to the reference model. The numbering displayed in the output grids will reflect the residue IDs in the reference model; for residues present in other chains but not the reference model, the residue ID will be displayed as "--", but the residue name(s) will still be shown. The primary limitation at this point is the lack of any direct link back to the original residue IDs, but the models displayed in Coot and PyMOL preserve these IDs.

Results

All output files will be written to a directory named `StructureComparison_X` (where X is the job ID). If structure superpositioning is enabled, there will be a separate PDB file for each chain, plus CCP4 map files if map coefficients were available. If the map coefficients were calculated from input X-ray data (not pre-calculated) these will also be written out.

When the application is finished running, a summary tab will show a list of all chains used for the analysis. The summary tab also lists basic statistics (such as number of atoms and residues, mean isotropic B-factor, ...). Additional tabs are added for each analysis. Each tab contains a grid with chains organized one per row, and residues of interest in columns. Color coding indicates the relationship of the specific residue in each cell to the equivalent residues, and/or properties such as validation outlier or multiple conformations. Cells colored in

grey indicated residues missing from the input chain.

The criteria used to display results vary among result tabs, but as the primary purpose of the program is to highlight regions of difference, homogeneous regions will usually be omitted from the grid.

The following analyses are carried out. It should be noted that the result tabs only appear if differences were found.

Ligand Summary: If ligands are present in at least one of the chains, they are listed in the results table, along with basic properties such as residue name, number of atoms and mean isotropic B-factor. Note that columns are sortable, f.ex. the table can be sorted according to the number of atoms. **Rotamers:** Any set of matching residues containing more than one rotamer type will be included, plus all outliers. **Coloring:** red for outliers, orange for "minority" conformations, green for "majority" conformations (at least 50% of chains under consideration), blue for multiple conformations, grey for missing side-chains or residues. The rotamer IDs are taken from the Ultimate Rotamer Library, which is described in Hintze et al. (2016). In the bottom part of the window is a summary table which can serve as a legend for the cell colors. **Ramachandran angles:** Any set of matching residues with different phi,psi angles plus any outliers will be displayed. Additionally, residues which fall into more than one distinct "favored" region will be included (this is somewhat redundant with the secondary structure comparison). **Coloring:** red for outliers, orange for "allowed" angles, green for "favored", blue for multiple conformations, grey for missing residues. **Missing atoms:** Any set of matching residues with at least one atom missing will be displayed. **Coloring:** red for residues missing backbone atoms (C, O, N, CA, or CB), orange for residues missing sidechain atoms, blue for entire missing residues, green for complete residues. The cell width can be expanded to read the list of atom names in the cell. **Secondary structure:** Any set of matching residues with more than one type of secondary structure assignment will be included. **Coloring:** white for unstructured/loop, green for alpha helices, orange for beta-sheets, blue for 3₁₀ helices, grey for missing residues. **Water locations:** Any water molecules not conserved in all chains are listed in the results table. Likewise, ions are included in the analysis. It is of particular interest to investigate areas with waters in some chains and ions at equivalent positions in other chains. Mixed ion/water sites are therefore listed at the leftmost part of the results table. **Coloring:** orange for ions, green for water molecules, white for sites without corresponding water molecule. **Ligand Information:** If ligands are present, their centers of mass are compared. The table is useful to assess if ligands are at equivalent positions in all chains. **Coloring:** blue for ligands, white for sites without ligand at the equivalent position. **Histidine Protonation:** Histidine side chains can have three different protonation states. If H atoms are present in at least one input chain, the results tab lists all histidine residues with different protonation states. **Omega Angles:** Omega angles are categorized as trans, cis or twisted. Any matching residues with different assignments are displayed in the results table. **B-factors:** Unlike the other analyses, this result does not identify heterogeneity on a per-residue basis, but instead plots the mean B-factor for each residue by chain, with the option of viewing only the main-chain or sidechain average. In addition to the absolute mean values, it can also display the values normalized to the overall mean for each chain, which may be useful when the structures being compared differ significantly in the scale of their B-factors. Points denoting rotamer and Ramachandran outliers will also be displayed. (Note that this plot does not interact directly with Coot or PyMOL.)

- **Ligand Summary:** If ligands are present in at least one of the chains, they are listed in the results table, along with basic properties such as residue name, number of atoms and mean isotropic B-factor. Note that columns are sortable, f.ex. the table can be sorted according to the number of atoms.

Ligand Summary: If ligands are present in at least one of the chains, they are listed in the results table, along with basic properties such as residue name, number of atoms and mean isotropic B-factor. Note that columns are sortable, f.ex. the table can be sorted according to the number of atoms.

- **Rotamers:** Any set of matching residues containing more than one rotamer type will be included, plus all outliers. **Coloring:** red for outliers, orange for "minority" conformations, green for "majority" conformations

(at least 50% of chains under consideration), blue for multiple conformations, grey for missing side-chains or residues. The rotamer IDs are taken from the Ultimate Rotamer Library, which is described in Hintze et al. (2016). In the bottom part of the window is a summary table which can serve as a legend for the cell colors.

Rotamers: Any set of matching residues containing more than one rotamer type will be included, plus all outliers. Coloring: red for outliers, orange for "minority" conformations, green for "majority" conformations (at least 50% of chains under consideration), blue for multiple conformations, grey for missing side-chains or residues. The rotamer IDs are taken from the Ultimate Rotamer Library, which is described in Hintze et al. (2016). In the bottom part of the window is a summary table which can serve as a legend for the cell colors.

- Ramachandran angles: Any set of matching residues with different phi,psi angles plus any outliers will be displayed. Additionally, residues which fall into more than one distinct "favored" region will be included (this is somewhat redundant with the secondary structure comparison). Coloring: red for outliers, orange for "allowed" angles, green for "favored", blue for multiple conformations, grey for missing residues.

Ramachandran angles: Any set of matching residues with different phi,psi angles plus any outliers will be displayed. Additionally, residues which fall into more than one distinct "favored" region will be included (this is somewhat redundant with the secondary structure comparison). Coloring: red for outliers, orange for "allowed" angles, green for "favored", blue for multiple conformations, grey for missing residues.

- Missing atoms: Any set of matching residues with at least one atom missing will be displayed. Coloring: red for residues missing backbone atoms (C, O, N, CA, or CB), orange for residues missing sidechain atoms, blue for entire missing residues, green for complete residues. The cell width can be expanded to read the list of atom names in the cell.

Missing atoms: Any set of matching residues with at least one atom missing will be displayed. Coloring: red for residues missing backbone atoms (C, O, N, CA, or CB), orange for residues missing sidechain atoms, blue for entire missing residues, green for complete residues. The cell width can be expanded to read the list of atom names in the cell.

- Secondary structure: Any set of matching residues with more than one type of secondary structure assignment will be included. Coloring: white for unstructured/loop, green for alpha helices, orange for beta-sheets, blue for 3₁₀ helices, grey for missing residues.

Secondary structure: Any set of matching residues with more than one type of secondary structure assignment will be included. Coloring: white for unstructured/loop, green for alpha helices, orange for beta-sheets, blue for 3₁₀ helices, grey for missing residues.

- Water locations: Any water molecules not conserved in all chains are listed in the results table. Likewise, ions are included in the analysis. It is of particular interest to investigate areas with waters in some chains and ions at equivalent positions in other chains. Mixed ion/water sites are therefore listed at the leftmost part of the results table. Coloring: orange for ions, green for water molecules, white for sites without corresponding water molecule.

Water locations: Any water molecules not conserved in all chains are listed in the results table. Likewise, ions are included in the analysis. It is of particular interest to investigate areas with waters in some chains and ions at equivalent positions in other chains. Mixed ion/water sites are therefore listed at the leftmost part of the results table. Coloring: orange for ions, green for water molecules, white for sites without corresponding water molecule.

- Ligand Information: If ligands are present, their centers of mass are compared. The table is useful to assess if ligands are at equivalent positions in all chains. Coloring: blue for ligands, white for sites without ligand at the equivalent position.

Ligand Information: If ligands are present, their centers of mass are compared. The table is useful to assess if ligands are at equivalent positions in all chains. Coloring: blue for ligands, white for sites without ligand at the equivalent position.

- Histidine Protonation: Histidine side chains can have three different protonation states. If H atoms are present in at least one input chain, the results tab lists all histidine residues with different protonation states.

Histidine Protonation: Histidine side chains can have three different protonation states. If H atoms are present in at least one input chain, the results tab lists all histidine residues with different protonation states.

- Omega Angles:: Omega angles are categorized as trans, cis or twisted. Any matching residues with different assignments are displayed in the results table.

Omega Angles:: Omega angles are categorized as trans, cis or twisted. Any matching residues with different assignments are displayed in the results table.

- B-factors: Unlike the other analyses, this result does not identify heterogeneity on a per-residue basis, but instead plots the mean B-factor for each residue by chain, with the option of viewing only the main-chain or sidechain average. In addition to the absolute mean values, it can also display the values normalized to the overall mean for each chain, which may be useful when the structures being compared differ significantly in the scale of their B-factors. Points denoting rotamer and Ramachandran outliers will also be displayed. (Note that this plot does not interact directly with Coot or PyMOL.)

B-factors: Unlike the other analyses, this result does not identify heterogeneity on a per-residue basis, but instead plots the mean B-factor for each residue by chain, with the option of viewing only the main-chain or sidechain average. In addition to the absolute mean values, it can also display the values normalized to the overall mean for each chain, which may be useful when the structures being compared differ significantly in the scale of their B-factors. Points denoting rotamer and Ramachandran outliers will also be displayed. (Note that this plot does not interact directly with Coot or PyMOL.)

Interacting with Coot

To show controls for sending data to and from Coot, click the icon on the toolbar of the results window. A small panel will appear:

Click the button labeled "Load models and maps" to read all structures into Coot simultaneously. Once this is complete they will appear in a checklist in Phenix; unchecking any structure will hide the associated objects in Coot. As with other validation programs in the Phenix GUI, clicking on the field for any residue in the results grid will recenter Coot on that residue (in the appropriate frame of reference depending on whether models were superposed). You may now adjust the models to match a consensus if desired.

If the structures were superposed, you should not try to use the models in Coot for further refinement. Instead, click the button labeled "Fetch modified structures" to write out the models from Coot to the results directory. Phenix will automatically recover the original orientation and replace the unmodified chain in the input structure with the model from Coot, while leaving all other atoms in place. The complete files will be written out to the result directory, renamed with the extension "_modified.pdb".

Reference

Coming soon.

Citations

Hintze BJ, Lewis SM, Richardson JS, Richardson DC. Molprobity's ultimate rotamer-library distributions for model validation. *Proteins-Struct. Funct. Bioinforma.* 2016 84:1177–1189. PubMed PMID: 27018641

Madhusudan, Akamine P, Xuong NH, Taylor SS. Crystal structure of a transition state mimic of the catalytic subunit of cAMP-dependent protein kinase. *Nat Struct Biol.* 2002 9:273-7. PubMed PMID: 11896404; PDB ID 1l3r

Wu J, Yang J, Kannan N, Madhusudan, Xuong NH, Ten Eyck LF, Taylor SS. Crystal structure of the E230Q mutant of cAMP-dependent protein kinase reveals an unexpected apoenzyme conformation and an extended N-terminal A helix. *Protein Sci.* 2005 14:2871-9. PubMed PMID: 16253959; PDB ID 1syk

Kim C, Xuong NH, Taylor SS. Crystal structure of a complex between the catalytic and regulatory (Rlalpha) subunits of PKA. *Science* 2005 307:690-6. PubMed PMID: 15692043; PDB ID 3fhi

Orts J, Tuma J, Reese M, Grimm SK, Monecke P, Bartoschek S, Schiffer A, Wendt KU, Griesinger C, Carlomagno T. Crystallography-independent determination of ligand binding modes. *Angew Chem Int Ed Engl.* 2008 47:7736-40. PubMed PMID: 18767090; PDB IDs 3dnd, 3dne

Thompson EE, Kornev AP, Kannan N, Kim C, Ten Eyck LF, Taylor SS. Comparative surface geometry of the protein kinase family. *Protein Sci.* 2009 18:2016-26. PubMed PMID: 19610074; PDB ID 3fjq

- Hintze BJ, Lewis SM, Richardson JS, Richardson DC. Molprobity's ultimate rotamer-library distributions for model validation. *Proteins-Struct. Funct. Bioinforma.* 2016 84:1177–1189. PubMed PMID: 27018641
- Madhusudan, Akamine P, Xuong NH, Taylor SS. Crystal structure of a transition state mimic of the catalytic subunit of cAMP-dependent protein kinase. *Nat Struct Biol.* 2002 9:273-7. PubMed PMID: 11896404; PDB ID 1l3r
- Wu J, Yang J, Kannan N, Madhusudan, Xuong NH, Ten Eyck LF, Taylor SS. Crystal structure of the E230Q mutant of cAMP-dependent protein kinase reveals an unexpected apoenzyme conformation and an extended N-terminal A helix. *Protein Sci.* 2005 14:2871-9. PubMed PMID: 16253959; PDB ID 1syk
- Kim C, Xuong NH, Taylor SS. Crystal structure of a complex between the catalytic and regulatory (Rlalpha) subunits of PKA. *Science* 2005 307:690-6. PubMed PMID: 15692043; PDB ID 3fhi
- Orts J, Tuma J, Reese M, Grimm SK, Monecke P, Bartoschek S, Schiffer A, Wendt KU, Griesinger C, Carlomagno T. Crystallography-independent determination of ligand binding modes. *Angew Chem Int Ed Engl.* 2008 47:7736-40. PubMed PMID: 18767090; PDB IDs 3dnd, 3dne
- Thompson EE, Kornev AP, Kannan N, Kim C, Ten Eyck LF, Taylor SS. Comparative surface geometry of the protein kinase family. *Protein Sci.* 2009 18:2016-26. PubMed PMID: 19610074; PDB ID 3fjq

structure_search

structure_search.html

Overview

Structure_search is a tool to quickly identify structural and/or sequence homologs of the input PDB file from the Protein Data Bank. It uses the SARST algorithm, and it's very fast. A typical search time against the whole PDB is usually less than one second. There is an option to allow users to obtain a list of ligands found in PDB structures of those homologs.

Usage

Identify and superpose homologous pdbs of mypdb.pdb
phenix.structure_search mypdb.pdb
obtain a list of homologs of mypdb.pdb and all ligands found in structures of those homologs
phenix.structure_search mypdb.pdb get_ligand=True
Use a local PDB mirror and obtain superposed homologs of mypdb.pdb
phenix.structure_search mypdb.pdb PDB_MIRRORDIR=/path/to/pdb_mirror/top-level

- Identify and superpose homologous pdbs of mypdb.pdb
phenix.structure_search mypdb.pdb

Identify and superpose homologous pdbs of mypdb.pdb

phenix.structure_search mypdb.pdb

- obtain a list of homologs of mypdb.pdb and all ligands found in structures of those homologs
phenix.structure_search mypdb.pdb get_ligand=True

obtain a list of homologs of mypdb.pdb and all ligands found in structures of those homologs

phenix.structure_search mypdb.pdb get_ligand=True

- Use a local PDB mirror and obtain superposed homologs of mypdb.pdb
phenix.structure_search mypdb.pdb PDB_MIRRORDIR=/path/to/pdb_mirror/top-level

Use a local PDB mirror and obtain superposed homologs of mypdb.pdb

phenix.structure_search mypdb.pdb PDB_MIRRORDIR=/path/to/pdb_mirror/top-level

More information can be found in input/Output files sections below:

Input files

required input:

pdb_file: the file containing the protein model of interest. By default, structure_search will search structure and sequence homologs of the input PDB file.

- pdb_file: the file containing the protein model of interest. By default, structure_search will search structure and sequence homologs of the input PDB file.

pdb_file: the file containing the protein model of interest.

By default, structure_search will search structure and sequence homologs of the input PDB file.

Optional inputs :

get_ligand:"=True" if want a list of ligands found in homologous PDBs, Default = False. job_title: current job title output_prefix: prefix for output files if needed. get_pdb:Collect and superpose the top N homologous pdbs

(default=10). `coot_display`: Display superposed pdb files in coot. [default=(False/True) as E-value(>/<)1E-18].
`sequence_only`:"=False" Perform only sequence search against PDB database using Phenix' internal DB.
`structure_only`: Perform only structure search against PDB database using the SARST method.
`PDB_MIRRORDIR`: Set option to use local PDB mirror instead of using RCSB server. See 'Using Local PDB mirror' section.

- `get_ligand`:"=True" if want a list of ligands found in homologous PDBs, Default = False.
- `job_title`: current job title
- `output_prefix`: prefix for output files if needed.
- `get_pdb`:Collect and superpose the top N homologous pdbs (default=10).
- `coot_display`: Display superposed pdb files in coot. [default=(False/True) as E-value(>/<)1E-18].
- `sequence_only`:"=False" Perform only sequence search against PDB database using Phenix' internal DB.
- `structure_only`: Perform only structure search against PDB database using the SARST method.
- `PDB_MIRRORDIR`: Set option to use local PDB mirror instead of using RCSB server. See 'Using Local PDB mirror' section.

Output files

In addition to screen output, these files contains results of `structure_search`:

`output_(sequence/structure).txt`: files containing sequence/structure homologs of 'pdb_file' sorted by scores.
`MyBlast_(sequence/structure).log`: Standard BLAST output with selected pairwise alignments. NOTE: for structure alignment, the 'sequences' are structure-based Ramachandran codes (see reference), not 1-letter code for amino acids. `pdb_ligand.txt` (if `get_ligand=True`): file containing all ligands found in all homologs from this search. superposed PDB files: Can be found in `TEMP_PDB_##` subdirectory as prompted in the program output.

- `output_(sequence/structure).txt`: files containing sequence/structure homologs of 'pdb_file' sorted by scores.
- `MyBlast_(sequence/structure).log`: Standard BLAST output with selected pairwise alignments. NOTE: for structure alignment, the 'sequences' are structure-based Ramachandran codes (see reference), not 1-letter code for amino acids.
- `pdb_ligand.txt` (if `get_ligand=True`): file containing all ligands found in all homologs from this search.
- superposed PDB files: Can be found in `TEMP_PDB_##` subdirectory as prompted in the program output.

Using Local PDB mirror

By default the program retrieves homologous PDB mmCIF files from RCSB server for downstream processes. Users may choose to use their local PDB mirror if environmental variables "`PDB_MIRRORDIR`" has already been defined in the running shell of `phenix.structure_search`. Alternatively, users may define it in the command-line script or specify the path in the GUI. See more details below.

`PDB_MIRRORDIR`: Defines the top level of the local PDB mirror. The program will try to retrieve PDB mmCIF files from local mirror unless the path does not exist. Note this assumes the directory tree under it follows that in the RCSB server and will try to access `$PDB_MIRRORDIR/data/structures/divided/mmCIF`. The program will fall back to using RCSB server should the path contain errors.

- **PDB_MIRRORDIR**: Defines the top level of the local PDB mirror. The program will try to retrieve PDB mmCIF files from local mirror unless the path does not exist. Note this assumes the directory tree under it follows that in the RCSB server and will try to access \$PDB_MIRRORDIR/data/structures/divided/mmCIF. The program will fall back to using RCSB server should the path contain errors.

References

Lo WC, Huang PJ, Chang CH, Lyu PC. BMC Bioinformatics. 2007, 8:307

List of all available keywords

structure_searchpdb_file = None Enter a PDB file name sequence = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB. output_prefix = 'output' Provide an output prefix if needed blastpath = None Enter path to blastall executable sequence_only = False Do a Blast search against PDBaa sequence instead of doing a Ramachandran-based structure search structure_only = False Do only a Ramachandran-based structure search. db_used = 'rcsb' structure database used in search. rcsb, scop95, or AF2db = 'rcsb' Database used in search. rcsb, scop95, or AF2.get_ligand = False Use get_ligand=True to retrieve ligands.get_ramcode_only = False Generate Rama code for input pdb/cif only. This is for developers only.get_xml_only = False Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only.use_pdb100aa = False Use PDB100 sequence database for sequence search.use_custom_db = False Use custom database specified by custom_db_files/custom_db_dir.custom_db_dir = None The directory of pdb/cif files to make custom database. Default is current directorycustom_db_files = None Filenames of the pdb/cif files separated by spaces for database. If none specified, all pdb/cif in the custom_db_dir will be collectedatom_selection = 'all' Choose part of the pdb used in the search (default=all). for example: chain B, resseq 113:219, ... etc. get_pdb = 10 get_pdb=N will collect and superpose the top N homologous pdbs. Use get_pdb=0 to disable this option.deposited_before = 0 Specify the latest year of matching structures to be considered for scoring. Pdb's deposited after this year will be discarded.deposited_after = 0 Specify the earliest year of matching structures to be considered for scoring. Pdb's deposited before this year will be discarded.batch_size = 0 Process the pdbs in batch of <batch_size> until <min_match> hits are identified or until all <get_pdb> pdbs are processedmin_match = 0 Finish structure_search when <min_match> matches are found. Usually uses with <trim_ends> to exit the search once find suitable pdbs.keep_all_pdb = False Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message.trim_ends = False Remove terminal residues of hit pdbs extending beyond those of the target pdb.write_pdb = True Set to False if no output pdb file is needed. Sometimes useful if use Structure_Search within another program and only want to pass pdb objects.write_results = True Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass pdb objects.trim_hit_pdb = False Remove extra domains, extended loops, and unfit portions of hit pdbs after superposed to the target pdb.pickle_hits = False Pickle blast hit results from xml output.coot_display = False (default) Display output pdb files in coot.ask_coot = True prompt for coot display optionsPDB_MIRRORDIR = None Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in PDB_MIRRORDIR/data/structures/divided/pdb directory. If you use PDB's rsync script, this variable would be the same as the \$MIRRORDIR set in the scriptPDB_MIRROR_MMCIFF = None Enter the parent directory of the mmCIF files in the local PDB mirror. MMCIFFs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be \$PDB_MIRRORDIR/data/structures/divided/mmCIF directory. PDB_MIRROR_PDB = None Enter the parent

directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory ## where ## are the second and third letters in the ...

- `structure_searchpdb_file` = None Enter a PDB file name sequence = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB. `output_prefix` = 'output' Provide an output prefix if needed `blastpath` = None Enter path to blastall executable `sequence_only` = False Do a Blast search against PDBaa sequence instead of doing a Ramachandran-based structure search `structure_only` = False Do only a Ramachandran-based structure search. `db_used` = 'rcsb' structure database used in search. `rcsb`, `scop95`, or `AF2db` = 'rcsb' Database used in search. `rcsb`, `scop95`, or `AF2`. `get_ligand` = False Use `get_ligand=True` to retrieve ligands. `get_ramacode_only` = False Generate Rama code for input pdb/cif only. This is for developers only. `get_xml_only` = False Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only. `use_pdb100aa` = False Use PDB100 sequence database for sequence search. `use_custom_db` = False Use custom database specified by `custom_db_files/custom_db_dir`. `custom_db_dir` = None The directory of pdb/cif files to make custom database. Default is current directory. `custom_db_files` = None Filenames of the pdb/cif files separated by spaces for database. If none specified, all pdb/cif in the `custom_db_dir` will be collected. `atom_selection` = 'all' Choose part of the pdb used in the search (default=all). for example: chain B, resseq 113:219, ... etc. `get_pdb` = 10 `get_pdb=N` will collect and superpose the top N homologous pdbs. Use `get_pdb=0` to disable this option. `deposited_before` = 0 Specify the latest year of matching structures to be considered for scoring. Pdb's deposited after this year will be discarded. `deposited_after` = 0 Specify the earliest year of matching structures to be considered for scoring. Pdb's deposited before this year will be discarded. `batch_size` = 0 Process the pdbs in batch of <batch_size> until <min_match> hits are identified or until all <get_pdb> pdbs are processed. `min_match` = 0 Finish structure_search when <min_match> matches are found. Usually uses with <trim_ends> to exit the search once find suitable pdbs. `keep_all_pdb` = False Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message. `trim_ends` = False Remove terminal residues of hit pdbs extending beyond those of the target pdb. `write_pdb` = True Set to False if no output pdb file is needed. Sometimes useful if use Structure_Search within another program and only want to pass pdb objects. `write_results` = True Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass pdb objects. `trim_hit_pdb` = False Remove extra domains, extended loops, and unfit portions of hit pdbs after superposed to the target pdb. `pickle_hits` = False Pickle blast hit results from xml output. `coot_display` = False (default) Display output pdb files in coot. `ask_coot` = True prompt for coot display options. `PDB_MIRROR_DIR` = None Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in `PDB_MIRROR_DIR/data/structures/divided/pdb` directory. If you use PDB's rsync script, this variable would be the same as the `$MIRROR_DIR` set in the script. `PDB_MIRROR_MMCIF` = None Enter the parent directory of the mmCIF files in the local PDB mirror. MMCIFs will be retrieved from subdirectory ## where ## are the second and third letters in the PDB id. This keyword should be `$PDB_MIRROR_DIR/data/structures/divided/mmcif` directory. `PDB_MIRROR_PDB` = None Enter the parent directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory ## where ## are the second and third letters in the ...

- `pdb_file` = None Enter a PDB file name

- `sequence` = None Optional Fasta sequence file. Only needed for a quick sequence search against RCSB without a PDB.

- `output_prefix` = 'output' Provide an output prefix if needed

- `blastpath = None` Enter path to blastall executable
- `sequence_only = False` Do a Blast search against PDBaa sequence instead of doing a Ramachandran-based structure search
- `structure_only = False` Do only a Ramachandran-based structure search.
- `db_used = 'rcsb'` structure database used in search. `rcsb`, `scop95`, or `AF2`
- `db = 'rcsb'` Database used in search. `rcsb`, `scop95`, or `AF2`.
- `get_ligand = False` Use `get_ligand=True` to retrieve ligands.
- `get_ramcode_only = False` Generate Rama code for input `pdb/cif` only. This is for developers only.
- `get_xml_only = False` Get BLAST XML output returned as a string object. No coordinate superposition will be performed. Developers only.
- `use_pdb100aa = False` Use PDB100 sequence database for sequence search.
- `use_custom_db = False` Use custom database specified by `custom_db_files/custom_db_dir`.
- `custom_db_dir = None` The directory of `pdb/cif` files to make custom database. Default is current directory
- `custom_db_files = None` Filenames of the `pdb/cif` files separated by spaces for database. If none specified, all `pdb/cif` in the `custom_db_dir` will be collected
- `atom_selection = 'all'` Choose part of the `pdb` used in the search (default=`all`). for example: chain B, resseq 113:219, ... etc.
- `get_pdb = 10` `get_pdb=N` will collect and superpose the top N homologous `pdb`s. Use `get_pdb=0` to disable this option.
- `deposited_before = 0` Specify the latest year of matching structures to be considered for scoring. `Pdb`s deposited after this year will be discarded.
- `deposited_after = 0` Specify the earliest year of matching structures to be considered for scoring. `Pdb`s deposited before this year will be discarded.
- `batch_size = 0` Process the `pdb`s in batch of `<batch_size>` until `<min_match>` hits are identified or until all `<get_pdb>` `pdb`s are processed
- `min_match = 0` Finish structure_search when `<min_match>` matches are found. Usually uses with `<trim_ends>` to exit the search once find suitable `pdb`s.
- `keep_all_pdb = False` Keep all the PDB files, including full PDB, PDB_Chain and superposed PDB_Chain. Default is False which will keep only superposed PDB_Chain files in the directory specified in the output message.
- `trim_ends = False` Remove terminal residues of hit `pdb`s extending beyond those of the the target `pdb`.
- `write_pdb = True` Set to False if no output `pdb` file is needed. Sometimes useful if use Structure_Search within another program and only want to pass `pdb` objects.
- `write_results = True` Set to False if no output results/log files is needed. Useful when calling Structure_Search within another program and only want to pass `pdb` objects.
- `trim_hit_pdb = False` Remove extra domains, extended loops, and unfit portions of hit `pdb`s after superposed to the target `pdb`.
- `pickle_hits = False` Pickle blast hit results from xml output.
- `coot_display = False` (default) Display output `pdb` files in coot.
- `ask_coot = True` prompt for coot display options

- `PDB_MIRRORDIR = None` Enter the top directory of local RCSB PDB mirror. The program will try to retrieve PDBs and/or structure factors from this mirror first. Note this assumes the directory trees under it follows those in RCSB -- pdb files as 'pdb####.ent.gz' in `PDB_MIRRORDIR/data/structures/divided/pdb` directory. If you use PDB's rsync script, this variable would be the same as the `$MIRRORDIR` set in the script
- `PDB_MIRROR_MMCF = None` Enter the parent directory of the mmCIF files in the local PDB mirror. MMCFs will be retrieved from subdirectory `##` where `##` are the second and third letters in the PDB id. This keyword should be `$PDB_MIRRORDIR/data/structures/divided/mmCIF` directory.
- `PDB_MIRROR_PDB = None` Enter the parent directory of the PDB files in the local PDB mirror. PDBs will be retrieved from subdirectory `##` where `##` are the second and third letters in the PDB id. This keyword should be `$PDB_MIRRORDIR/data/structures/divided/pdb` directory. We recommend setting `PDB_MIRRORDIR` and it will take care of both `PDB_MIRROR_PDB` and others together. However, users may choose to specify `PDB_MIRROR_PDB` directly
- `PDB_MIRROR_STRUCTURE_FACTORS = None` Enter the parent directory of the PDB files in the local PDB mirror. structure factors s will be retrieved from subdirectory `##` where `##` are the second and third letters in the PDB id. This keyword should be the same as the `$PDB_MIRRORDIR/data/structures/divided/structure_factors` directory. We recommend setting `PDB_MIRRORDIR` and it will take care of both `PDB_MIRROR_PDB` and `PDB_MIRROR_STRUCTURE_FACTORS` together. However, users may choose to specify `PDB_MIRROR_STRUCTURE_FACTORS` directly
- `local_pdb_dir = None` Enter the path directly to your local PDB repository.
- `verbose = False` verbose output
- `debug = False` debugging output
- `job_title = None` Job title in PHENIX GUI, not used on command line
- `guiGUI-specific parameter required for output directory` `output_dir = None`
- `output_dir = None`

Welcome to AI-powered Phenix and CCTBX programming!

test.html

A new mindset for programming: Think about what you want to accomplish and ask your AI assistants to help you write the code to do it.

Why use AI: The Phenix developer AI chatbot has all the Phenix and CCTBX documentation and code in its sources. It can help you get going quickly. It can even write unit tests for you.

Be aware: AI chatbots usually do not get everything right on the first try. It is your job to tell them what is wrong (for starters, paste in a traceback!) and to make your code work just the way you want it to.

Use the right AI tool for the job: Use the Phenix developer chat to write Phenix and CCTBX code. It knows the code base and how to use the Program Template. Use Gemini or other general-purpose AI chatbots to write scripts that do not use Phenix or CCTBX tools. These chatbots are better at programming but they do not know about the Phenix code base.

Help chatbots

- Phenix developer chat. (Private, do not share link). Use this for Phenix or CCTBX programming questions or to write Python scripts in Phenix/CCTBX. Designed to allow easy creation of ProgramTemplate files.
- CCTBX developer chat. Use this for CCTBX programming questions or scripts.
- Phenix user help chat. Use this for questions on how to run Phenix or for general questions about crystallography and cryo-EM structure determination.

Also see some helpful hints on using AI chatbots.

Programming documentation

- CCTBX developer documentation Comprehensive guide to using CCTBX tools. High-level cctbx managers, maps, and low-level cctbx objects and tools. Includes tutorials, sample scripts, how to test your code, and many helpful hints on using CCTBX tools
- Phenix developer documentation How to use the Program Template, how to use the model-building tools in Phenix, handy restraint jiffies, Phenix/CCTBX high-level reference guide.
- PHENIX/CCTBX Programming API Fully indexed documentation of all CCTBX and Phenix Python classes and methods. Includes all code from CCTBX and from Phenix. You can update this on your machine with the command:

```
phenix.rebuild_phenix_api
```

- CCTBX Programming API Fully indexed documentation and code for all CCTBX Python classes and methods. You can update this on your machine with the command:

```
phenix.rebuild_cctbx_api
```

Analyzing unmerged data in Phenix

unmerged_data.html

Introduction

Most of the programs in Phenix assume that the input data, regardless of type or format, are already both scaled and merged to contain only unique reflections. The major exception is AutoSol, which can accept certain unmerged files (for a more limited set of formats, primarily Scalepack) and performs its own local scaling. In other situations, some programs will automatically merge the data themselves if necessary, discarding any information about the variance of individual observations.

A note on terminology: "unmerged" in this context means that the original individual observations are kept separate after scaling, including symmetry-related reflections as well as multiple observations of the same h,k,l . This is distinct from "anomalous" data, where the Friedel mates h,k,l and $-h,-k,-l$ are not merged, but each of these is (usually) still the average of multiple observations. Although many programs in Phenix deal with anomalous data, they are not useful for calculating merging statistics.

Several utilities are available in Phenix which use unmerged intensities to calculate various data quality-related metrics. These include Xtriage, which will display standard merging statistics; the standalone version of this routine, `phenix.merging_statistics`; a separate utility for calculating CC^* (Karplus & Diederichs 2012), `phenix.cc_star`; and the Table 1 program. Their use as it relates to unmerged data is described below. Note that due to implementation details, small numerical differences (up to a few tenths of a percent) may occur between the originally reported statistics in the processing logfiles, and the numbers reported by Phenix.

Important: we have provided a list of statistics with pointers to the relevant publications in the "Details" section, but this document does not attempt to explain the mathematics or logic behind the statistics calculated here. We strongly recommend that you read the listed references; the publication(s) associated with the data processing software used may also be useful (see for example Evans (2011)).

If you want to experiment with these programs and do not have any data of your own to use, the Phenix installation includes an unmerged data file in the p9-sad example:

```
$PHENIX/examples/p9-sad/p9_se_w2.sca
```

The format does not contain complete symmetry information (and the space group is specified incorrectly), but you can use the symmetry information in `p9.pdb` in the same directory.

Obtaining unmerged data

The procedure differs between various data-processing software; these are the methods we are familiar with:

xia2: the DataFiles directory will contain a file in Scalepack format, ending in `"_unmerged.sca"`. No additional steps are required. aimless (CCP4): the OUTPUT keyword has an UNMERGED option. XDS: the file named `XDS_ASCII.HKL` output by CORRECT will contain unmerged data. If you are using XSCALE to scale data from multiple crystals, add the keyword `MERGE=FALSE` to the input script. HKL2000: in the Macros tab, in the Scaling section type "no merge original index", then re-run scaling (with a different file name than you used for merged data). This is currently incompatible with the "automatic correction" option, which will always output a merged file.

- xia2: the DataFiles directory will contain a file in Scalepack format, ending in `"_unmerged.sca"`. No additional steps are required.

- aimless (CCP4): the OUTPUT keyword has an UNMERGED option.
- XDS: the file named XDS_ASCII.HKL output by CORRECT will contain unmerged data. If you are using XSCALE to scale data from multiple crystals, add the keyword MERGE=FALSE to the input script.
- HKL2000: in the Macros tab, in the Scaling section type "no merge original index", then re-run scaling (with a different file name than you used for merged data). This is currently incompatible with the "automatic correction" option, which will always output a merged file.

Note that unmerged Scalepack format does not include the unit cell, so symmetry needs to be supplied from a different source (this is handled automatically in many of the programs below). MTZ and XDS formats include complete symmetry. Not all files in these formats are necessarily suitable: in particular, note that MTZ files output by Mosflm have not had scaling performed yet.

Xtriage and phenix.merging_statistics

phenix.merging_statistics is a simple program to calculate the standard statistics output by most data processing software, such as mean(I/sigma), redundancy, R-merge, R-meas (Diederichs & Karplus 1997), R-pim (Weiss 2001), CC1/2, and others. The input may be in any format, including unmerged XDS, unmerged Scalepack, multi-batch MTZ, or SHELX (the latter will only work on the command line, however). The number of resolution bins is set to 10 by default, but is adjustable by the n_bins parameter. All data will be used unless the high_resolution or low_resolution parameters are set; the program does not attempt to identify an appropriate high-resolution cutoff on its own. Both a command-line tool and a GUI (pictured above) are available. The output in either case is a table of statistics by resolution bin:

```
Resolution: 28.53 - 1.75
Observations: 145453
Unique reflections: 21557
Redundancy: 6.7
Completeness: 99.92%
Mean intensity: 18672.0
Mean I/sigma(I): 14.9
R-merge: 0.073
R-meas: 0.079
R-pim: 0.030
```

Redundancies (non-anomalous):

```
1 : 50
2 : 163
3 : 597
4 : 1766
5 : 1869
6 : 3731
7 : 5512
8 : 5723
9 : 1526
10 : 490
11 : 74
12 : 53
13 : 3
```

Statistics by resolution bin:

d_max	d_min	#obs	#uniq	mult.	%comp	<I>	<I/si>	r_mrg	r_meas	r_pim	cc1/2
28.53	3.77	15699	2254	6.96	99.87	78997.8	23.4	0.061	0.066	0.025	0.997
3.77	2.99	15703	2182	7.20	99.95	47400.1	23.1	0.061	0.066	0.024	0.997
2.99	2.61	15641	2172	7.20	100.00	17930.9	21.1	0.074	0.080	0.030	0.996
2.61	2.37	15309	2138	7.16	100.00	10520.1	18.6	0.090	0.097	0.036	0.995
2.37	2.21	15044	2146	7.01	99.95	9103.8	17.2	0.093	0.101	0.038	0.995
2.20	2.07	14571	2145	6.79	100.00	6560.2	13.5	0.108	0.117	0.045	0.993
2.07	1.97	13973	2135	6.54	100.00	5016.1	10.8	0.121	0.131	0.051	0.992
1.97	1.89	13540	2141	6.32	100.00	3620.6	8.6	0.145	0.158	0.062	0.984
1.88	1.81	13010	2104	6.18	99.95	2070.5	6.8	0.197	0.215	0.085	0.980

```
1.81 1.75 12963 2140 6.06 99.49 1477.4 5.6 0.247 0.270 0.108 0.970
28.53 1.75 145453 21557 6.75 99.92 18672.0 14.9 0.073 0.079 0.030 0.998
```

In the GUI version, plots are also available:

Xtriage (Zwart et al. 2005) will also calculate these statistics if provided with unmerged intensities; in the GUI, these will appear in an extra tab before the data quality analyses.

phenix.cc_star

This is an implementation of the method outlined in Karplus & Diederichs (2012) for determining the optimal resolution cutoff, as an alternative to the traditional criteria such as limits on R-merge or I/sigma. It is essentially a combination of phenix.merging_statistics with the recalculation of R-factors for the model. The statistics CC* is an estimate of the "true" CC of the data under examination to the (unknown) true intensities. This places it on the same scale as CC(work) and CC(free) for the model and data, and the relationship between these statistics can be used as a guide for truncating the data.

Required inputs are unmerged intensities, merged data (either amplitudes or intensities - if the latter, French&Wilson treatment will be performed), R-free flags, and either a pre-calculated F(model) array, which is output by programs like phenix.refine, or the current refined model. The results should be essentially the same in either case, but supplying F(model) directly will run faster. The statistics overlap with those reported by the merging program, but with the addition of R-factors and CCs for model/data agreement.

phenix.table_one

This utility generates the standard Table 1 of crystallographic statistics found in most structure publications. If unmerged data are included along with the merged data and R-free flags, the intensity merging statistics will be calculated from these instead of parsing the data processing logfiles. We recommend that you use the unmerged data if available, as they are more reliably parsed than logfiles.

Details for individual statistics

Note that the programs listed above display slightly different subsets of these statistics.

Mean(I/signal): the signal-to-noise ratio. Traditionally the data are often cut off at the resolution where this statistic declines to 2, but this may exclude genuine measurements.

Redundancy or Multiplicity: the average number of observations of each reflection.

R-merge or R-symm: the simple merging R-factor for the multiple observations. Ideally this should be as low as possible, but for reasons discussed in the listed references, using it as a resolution cutoff is problematic. (Note that it will always increase significantly in the high-resolution shells.)

R-meas or R-rim: an alternative to R-merge which is intended to be independent of redundancy. See Diederichs & Karplus (1997) for explanation.

R-pim: another alternative to R-merge, intended to show the precision of the measurements. See Weiss (2001).

CC1/2: the correlation of one half of the observations (chosen at random, but with approximately even distribution for each reflection) to the other half. A similar statistic is used in electron microscopy; see Karplus & Diederichs (2012) for more information.

CC*: a modification of CC1/2, intended to show the true correlation of the observed data to the unknown "true" intensities. Used in conjunction with CC(work) and CC(free). See Karplus & Diederichs (2012).

CC(work) and CC(free): correlation of the observed intensities to the model-based intensities, for the work and test sets respectively. Should be on the same scale as CC*.

References

Improved R-factors for diffraction data analysis in macromolecular crystallography. K. Diederichs, and P.A. Karplus. Nat Struct Biol 4, 269-75 (1997).

- Improved R-factors for diffraction data analysis in macromolecular crystallography. K. Diederichs, and P.A. Karplus. Nat Struct Biol 4, 269-75 (1997).

An introduction to data reduction: space-group determination, scaling and intensity statistics. P.R. Evans. Acta Crystallogr D Biol Crystallogr 67, 282-92 (2011).

- An introduction to data reduction: space-group determination, scaling and intensity statistics. P.R. Evans. Acta Crystallogr D Biol Crystallogr 67, 282-92 (2011).

Linking crystallographic model and data quality. P.A. Karplus, and K. Diederichs. Science 336, 1030-1033 (2012).

- Linking crystallographic model and data quality. P.A. Karplus, and K. Diederichs. Science 336, 1030-1033 (2012).

Global indicators of data quality. M.S. Weiss. J. Appl. Cryst. 34, 130-135 (2001).

- Global indicators of data quality. M.S. Weiss. J. Appl. Cryst. 34, 130-135 (2001).

Xtriage and Fest: automatic assessment of X-ray data and substructure structure factor estimation. P.H. Zwart, R.W. Grosse-Kunstleve, and P.D. Adams. CCP4 Newsletter Winter, Contribution 7 (2005).

- Xtriage and Fest: automatic assessment of X-ray data and substructure structure factor estimation. P.H. Zwart, R.W. Grosse-Kunstleve, and P.D. Adams. CCP4 Newsletter Winter, Contribution 7 (2005).

Using AI to help generate doc strings

using_ai_for_doc_strings.html

Documenting your code with doc strings or other helpful information makes it easier for you and others to understand, use, and modify your code later.

Additionally, doc strings can help AI tools identify what code would be useful in a new situation and to document the way to use your code.

libtbx.add_docstrings_with_ai: a tool to automatically add doc strings

There are many ways to add doc strings to your code, but even most AI tools will not add doc strings to large files. The CCTBX tool `libtbx.add_docstrings_with_ai` will break up a file into pieces, send each one to gemini-1.5 for doc string generation, catch problems and resend, and assemble the result into a new file that has all the coding lines from the original file and a new set of documentation strings.

Here is an example. This is code supplied to `libtbx.add_docstrings_with_ai`:

```
class xh_connectivity_table2(object):
    def __init__(self, geometry, xray_structure):
        bond_proxies_simple, asu = geometry.geometry.get_all_bond_proxies(
            sites_cart=xray_structure.sites_cart())
        scatterers = xray_structure.scatterers()
```

The AI tool analyzes this, figures out something about what it is doing, and summarizes the arguments and any return values:

```
class xh_connectivity_table2(object):
    """
    Stores X-H connectivity information, including angle information.
    """
    def __init__(self, geometry, xray_structure):
        """
        Initialize the X-H connectivity table.

        Args:
        geometry (mmtbx.geometry_restraints.manager): Geometry restraints manager.
        xray_structure (cctbx.xray.structure.structure): X-ray structure object.
        """
        bond_proxies_simple, asu = geometry.geometry.get_all_bond_proxies(
            sites_cart=xray_structure.sites_cart())
        scatterers = xray_structure.scatterers()
        ...
```

Preparing to run libtbx.add_docstrings_with_ai

To run this tool, you will need a Google API key, as the tool talks to Google directly. You can get a free key like this:

Go to Google AI Studio: Navigate to makersuite.google.com/app/apikey. You may need to sign in with your Google account. Create a New Key: Click the button labeled "Create API key in new project". Copy Your Key: A pop-up window will appear displaying your new API key. Click the copy icon next to the key to copy it to your clipboard. Set the `GOOGLE_API_KEY` environment variable: `csh: setenv GOOGLE_API_KEY <your-api-key>` `bash: set GOOGLE_API_KEY=<your-api-key>`

Go to Google AI Studio: Navigate to makersuite.google.com/app/apikey. You may need to sign in with your Google account.

Create a New Key: Click the button labeled "Create API key in new project".

Copy Your Key: A pop-up window will appear displaying your new API key. Click the copy icon next to the key to copy it to your clipboard.

Set the GOOGLE_API_KEY environment variable:

csh: setenv GOOGLE_API_KEY <your-api-key> bash: set GOOGLE_API_KEY=<your-api-key>

How to run libtbx.add_docstrings_with_ai

You can run the tool with:

```
libtbx.add_docstrings_with_ai input_dir output_dir
```

You put the files you want documented in input_dir and it will write them to output_dir (creating output_dir if necessary).

Be safe: check the results

While this tool is designed to only change doc strings, you should carefully review the new version of each of your files. You can use

```
diff input_dir/my_file.py output_dir/my_file.py
```

to see what is changed, for example. Also just have a look at each new file.

Helpful hints for using AI in coding for Phenix and CCTBX

[using_ai_in_programming.html](#)

What if you could write pretty good code without knowing every detail of the tools you are going to use?

AI tools are great for creating scripts that do something pretty close to what you ask for. Then if you have a way of noticing what is not perfect about the results, you can iteratively ask the AI tools to fix your scripts. This can be an incredibly powerful, and relatively easy, process even if the AI is using tools that you do not know a lot about. The AI tools are great at explaining what the code is doing as well, so it can be great for learning how to use available tools.

How to be safe using an AI chatbot for coding

While AI chatbots are not malicious, they can write code that does not do what you intend.

Look through the code that a chatbot suggests before running it. It is your responsibility to be able to recognize whether the code is what you intend or not.

Make sure that the code does not accidentally delete or modify files or carry out procedures that you do not want.

Code from chatbots may require installation of new packages. Before installing any package, look it up and make sure that it is a mainstream package.

How to write code with an AI chatbot

First, the AI has to have been trained on programming with the tools that you need to use. For general scripting, tools such as Gemini and ChatGPT are already trained on most common tools, so you are all set. For writing scripts in a specialized environment such as Phenix or CCTBX, your AI tool may need to have imported the codebase that you are going to base your work on. And no worries: there are already AI tools all set up to work with CCTBX or with Phenix and CCTBX.

Second, you need to communicate clearly what you want the AI chatbot to create. If you want a script written in a certain way or with certain tools, you need to tell it these things. A good way to do this is to have a discussion with the AI tool, asking it how to go about carrying out the task you want, choosing a path that you like, and asking it to write a script to do the task in that way.

Third, and very importantly, you have to have a way of determining whether the code that the AI tool creates is actually doing what you want. There are several ways to do this. One is to specify just what it is supposed to do up front, and then take the script that the AI produces for you, run it, and see if it does what you want. Another is to ask the AI to create a set of unit tests for the script. Then you can look through the unit tests and make sure they are what you want and that they all pass.

Fourth, you have to expect that the AI will not get it right the first time. The amazing thing is that it is often quite close. If you run the script that the AI creates and it crashes, you can just paste the traceback text in to the AI chat and in most cases it will try to figure out what it did wrong and it will create an updated script fixing that problem. You can do this to fix anything that you don't like. If the formatting of your output is not to your liking, just ask the AI to change it. If you would like more input parameters, just ask it to add them. While this process is a little tedious, it is very often a lot faster than writing a script by yourself. Once the AI version of your script is pretty close to what you want, you can also just edit it yourself to make it do just what you want.

How to use the Phenix developer AI chatbot to write a ProgramTemplate script

While AI chatbots are quite amazing at writing code, they are (so far) not that great at following instructions to the letter. Consequently writing very specialized code that has to follow specific formatting is sometimes a little difficult with AI.

One way to get around this is to first specify everything that you want the AI chatbot to consider, then ask it to write code, then repeat instructions that the chatbot did not follow correctly and ask it to fix the code accordingly.

In the Phenix developer AI chatbot this process can be carried out when you want to write a script that uses the ProgramTemplate, the standard Phenix script for gathering information from a user and running a program. An important element of this is that the chatbot has already been pre-loaded with a source that tells it the rules for the ProgramTemplate. This source contains the text "Chatbot Program Template", and you can tell the chatbot to focus on the contents of this source.

Here is how you do this:

```
First chat with the AI bot about what you want to do and decide which
of the possibilities that the AI gives you that you want to follow.
```

```
Then, supposing you want to write a script to calculate MolProbity
scores, type a command like this:
```

```
1. Describe in detail all the instructions in the source that contains
the exact text: "Chatbot Program Template". 2. Following these
instructions, write a script that reads in model.pdb and prints
out its molprobity score."
```

Now the AI will try to follow the Program Template rules and write you a script. Although it may not be perfect, it is likely to be close. Then if it does not run, you can paste in the traceback or incorrect outputs and let it figure out what it did wrong. If there are specific things that you notice that it did wrong, you can point those out and ask for a new script.

Notes

When the AI chatbots write code, they print it out on the screen and usually also supply a button to copy the whole code snippet. Make sure you wait until the entire code snippet is written to the screen before hitting the copy button or you will get an incomplete script.

phenix.varref: a tool for refinement of a model against multiple maps

varref.html

Contents

- Description
- Availability
- Contact
- Command line examples
- Notes
- References

Description

phenix.varref stands for 'variability refinement'. The program refines a model into an ensemble of maps derived from cryo-EM. The ensemble of maps represents the sample's heterogeneity captured during the experiment. The inputs are typically the refined model against the consensus high-resolution map, and the map series derived from 3DVA (cryosparc), or multibody refinement (Relion) or equivalent. The result of refinement with phenix.varref is an ensemble of real-space refined models that adequately represents the ensemble of input maps. For more details refer to: <http://dx.doi.org/10.1016/j.bbamem.2023.184133>.

Availability

The program is available in the command line and in the GUI.

Contact

For questions, bug reports, feature requests: help@phenix-online.org

IMPORTANT: any problem report should be sent after trying the latest nightly build of Phenix:

https://www.phenix-online.org/download/nightly_builds.cgi

Bug or problem reports should include at least 1) all input files used to run the program, 2) list of parameters that were customized, 3) error message and 4) a brief description of steps that lead to the problem. Please use file sharing tools (such as Dropbox, Google Drive, etc) to send large files. All files sent to us are handled confidentially and only used to solve the reported issue, and discarded afterwards.

Command line examples

A typical run: `phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \ nproc=10 models_per_map=50` The program takes one model and list of maps in any order. The resolution is a mandatory parameter. nproc specifies the number of processors available to the program to use. A run using a single processor can take a very long time! Use as many CPU as possible! models_per_map specifies the size of the ensemble to be refined against each individual map. The output ensemble accumulates overall best refined models taken from ensembles of models refined against each individual map. The model contains an unknown to Phenix ligand: `phenix.varref model.pdb ligand.cif map1.mrc map2.mrc... mapN.mrc \`

resolution=4.2 nproc=10 models_per_map=50 The ligand dictionary CIF file can be obtained using ReadySet! or eLBOW programs of Phenix and supplied to phenix.varref as an additional file. Anchoring solvent, ligands and other small molecules to the macromolecule: phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \ nproc=10 models_per_map=50 parameters.eff phenix.varref uses a rather aggressive refinement technique such as morphing, simulated annealing and perturbing initial model by means of MD. This means that small molecule (non-macromolecules) entities of the model may drift away from their original position or binding sites. To prevent this from happening they can be anchored to the macromolecule using custom bonds (as well as angles and so on). Refer to phenix.refine documentation for more details. In the example above this is done by providing a parameter file containing the following lines: geometry_restraints { edits { bond { atom_selection_1 = chain B and resseq 87 and name N atom_selection_2 = chain S and resseq 1 and resname HOH and name O distance_ideal = 2.0 sigma = 0.01 } } } Here the water oxygen in chain S (residue number 1) is linked to the Nitrogen of residue 87 in chain B with the bond length of 2Å. Any number of bonds can be specified.

- A typical run: phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \ nproc=10 models_per_map=50 The program takes one model and list of maps in any order. The resolution is a mandatory parameter. nproc specifies the number of processors available to the program to use. A run using a single processor can take a very long time! Use as many CPU as possible! models_per_map specifies the size of the ensemble to be refined against each individual map. The output ensemble accumulates overall best refined models taken from ensembles of models refined against each individual map.

A typical run:

```
phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \
nproc=10 models_per_map=50
```

The program takes one model and list of maps in any order. The resolution is a mandatory parameter. nproc specifies the number of processors available to the program to use. A run using a single processor can take a very long time! Use as many CPU as possible! models_per_map specifies the size of the ensemble to be refined against each individual map. The output ensemble accumulates overall best refined models taken from ensembles of models refined against each individual map.

- The model contains an unknown to Phenix ligand: phenix.varref model.pdb ligand.cif map1.mrc map2.mrc... mapN.mrc \ resolution=4.2 nproc=10 models_per_map=50 The ligand dictionary CIF file can be obtained using ReadySet! or eLBOW programs of Phenix and supplied to phenix.varref as an additional file.

The model contains an unknown to Phenix ligand:

```
phenix.varref model.pdb ligand.cif map1.mrc map2.mrc... mapN.mrc \
resolution=4.2 nproc=10 models_per_map=50
```

The ligand dictionary CIF file can be obtained using ReadySet! or eLBOW programs of Phenix and supplied to phenix.varref as an additional file.

- Anchoring solvent, ligands and other small molecules to the macromolecule: phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \ nproc=10 models_per_map=50 parameters.eff phenix.varref uses a rather aggressive refinement technique such as morphing, simulated annealing and perturbing initial model by means of MD. This means that small molecule (non-macromolecules) entities of the model may drift away from their original position or binding sites. To prevent this from happening they can be anchored to the macromolecule using custom bonds (as well as angles and so on). Refer to phenix.refine documentation for more details. In the example above this is done by providing a parameter file containing the following lines: geometry_restraints { edits { bond { atom_selection_1 = chain B and

resseq 87 and name N atom_selection_2 = chain S and resseq 1 and resname HOH and name O distance_ideal = 2.0 sigma = 0.01 } } } Here the water oxygen in chain S (residue number 1) is linked to the Nitrogen of residue 87 in chain B with the bond length of 2Å. Any number of bonds can be specified.

Anchoring solvent, ligands and other small molecules to the macromolecule:

```
phenix.varref model.pdb map1.mrc map2.mrc... mapN.mrc resolution=4.2 \
nproc=10 models_per_map=50 parameters.eff
```

phenix.varref uses a rather aggressive refinement technique such as morphing, simulated annealing and perturbing initial model by means of MD. This means that small molecule (non-macromolecules) entities of the model may drift away from their original position or binding sites. To prevent this from happening they can be anchored to the macromolecule using custom bonds (as well as angles and so on). Refer to phenix.refine documentation for more details. In the example above this is done by providing a parameter file containing the following lines:

```
geometry_restraints {
  edits {
    bond {
      atom_selection_1 = chain B and resseq 87 and name N
      atom_selection_2 = chain S and resseq 1 and resname HOH and name O
      distance_ideal = 2.0
      sigma = 0.01
    }
  }
}
```

Here the water oxygen in chain S (residue number 1) is linked to the Nitrogen of residue 87 in chain B with the bond length of 2Å. Any number of bonds can be specified.

Notes

The program can use multiple CPU. The program does not use GPU. Running on single processor can be very slow. Use as many CPUs as possible.

- The program can use multiple CPU. The program does not use GPU. Running on single processor can be very slow. Use as many CPUs as possible.

References

Afonine PV, Gobet A, Moissonnier L, Martin J, Poon BK, Chaptal V. Conformational space exploration of cryo-EM structures by variability refinement. *Biochim Biophys Acta Biomembr.* 2023 Apr;1865(4):184133. doi: 10.1016/j.bbamem.2023.184133. Epub 2023 Feb 3. PMID: 36738875.

- Afonine PV, Gobet A, Moissonnier L, Martin J, Poon BK, Chaptal V. Conformational space exploration of cryo-EM structures by variability refinement. *Biochim Biophys Acta Biomembr.* 2023 Apr;1865(4):184133. doi: 10.1016/j.bbamem.2023.184133. Epub 2023 Feb 3. PMID: 36738875.

Structure factor file manipulations with Xmanip

xmanip.html

Author(s)

- Xmanip: Peter Zwart
- Phil command interpreter: Ralf W. Grosse-Kunstleve

Purpose

Manipulation of reflection data and models

Usage

Command line interface

xmanip can be invoked via the command line interface with instructions given in a specific definition file:

```
phenix.xmanip params.def
```

The full set of definitions can be obtained by typing:

```
phenix.xmanip
```

which results in:

```
xmanip {
  input {
    unit_cell = None
    space_group = None
    xray_data {
      file_name = None
      labels = None
      label_appendix = None
      name = None
      write_out = None
    }
    model {
      file_name = None
    }
  }
  parameters {
    action = reindex manipulate_pdb *manipulate_miller
    reindex {
      standard_laws = niggli *reference_setting invert user_supplied
      user_supplied_law = "h,k,l"
    }
    manipulate_miller {
      task = get_dano get_diso lsq_scale sfcalc *custom None
      output_label_root = "FMODEL"
      get_dano {
        input_data = None
      }
      get_diso {
        native = None
        derivative = None
        use_intensities = True
        use_weights = True
        scale_weight = True
      }
    }
  }
}
```

```

lsq_scale {
input_data_1 = None
input_data_2 = None
use_intensities = True
use_weights = True
scale_weight = True
}
sfcalc {
fobs = None
output = *2mFo-DFc mFo-DFc complex_fcalc abs_fcalc intensities
use_bulk_and_scale = *as_estimated user_applied
bulk_and_scale_parameters {
d_min = 2
overall {
b_cart {
b_11 = 0
b_22 = 0
b_33 = 0
b_12 = 0
b_13 = 0
b_23 = 0
}
k_overall = 0.1
}
solvent {
k_sol = 0.3
b_sol = 56
}
}
custom{
code = print &gt;&gt; out, "hello world"
}
}

manipulate_pdb{
task = apply_operator *set_b
apply_operator{
operator = "x,y,z"
invert=False
concatenate_model=False
chain_id_increment=1
}
set_b{
b_iso = 30
}
}
output {
logfile = "xmanip.log"
hklout = "xmanip.mtz"
xyzout = "xmanip.pdb"
}
}

```

Detailed explanation of the scopes follow below.

Parameters and definitions

The xmanip.input scope defines which files and which data xmanip reads in:

```

input {
unit_cell = None # unit cell. Specify when not in reflection or pdb files
space_group = None # space group. Specify when not in reflection or pdb files
xray_data {
file_name = None # File from which data will be read
}
}

```

```

labels = None # Labels to read in.
label_appendix = None # Label appendix: when writing out the new mtz file, this
appendix will be added to the current label.
name = None # A data set name. Useful for manipulation
write_out = None # Determines if this data set will be written to the final
mtz file
}
model {
file_name = None # An input pdb file
}
}

```

One can define as many sub-scopes of `xray_data` as desired (see examples). The specific tasks of `xmanip` are controlled by the `xmanip.parameters.action` key. Possible options are:

- `reindex`
- `manipulate_pdb`
- `manipulate_miller`

Reindexing: reindexing of a data set (and a model) is controlled by the `xmanip.parameters.reindex` scope. Standard laws are available:

- `niggli`: Brings unit cell to the niggli setting.
- `reference_setting`: Brings space group to the reference setting
- `invert`: Inverts a data set
- `user_supplied`: A user supplied reindexing law is used, specified by `reindex.user_supplied_law`

`manipulate_pdb`: A pdb file can be modified by either applying a symmetry operator to the coordinates (select the `apply_operator` task from the `manipulate_pdb.task` list. The operator needs to be specified by `apply_operator.operator`. Setting `apply_operator.invert` to true will invert the supplied operator. One can choose to put out the newly generated chain with the original chain (set `concatenate_model = True`). The new chain ID can be controlled with the `chain_id_increment` parameter. `manipulate_miller`: Reflection data can be manipulate in various ways:

- `get_dano`: Get anomalous differences from the data set with name specified by `manipulate_miller.get_dano.input_data`.
- `get_diso`: Get isomorphous differences (derivative-native) from the data sets specified by the names `manipulate_miller.get_diso.native` and `manipulate_miller.get_diso.derivative`. Least squares scaling of the derivative to the native can be done on intensities (`use_intensities=True`), with or without using sigmas (`use_weights`) and by scaling the weights if desired (recommended).
- `lsq_scale`: As above, no isomorphous difference are computed, only `input_data_2` is scaled and returned.
- `sfcalc`: Structure factor calculation. Requires a pdb file to be read in. Possible output coefficients are
- `2mFo-DFc` (Fobs required. specify `sfcalc.fobs`).
- `mFo-DFc` (Fobs required. specify `sfcalc.fobs`).
- `complex_fcalc` (FC,PHIC)
- `abs_fcalc` (FC)
- `intensities` (FC²)

bulk solvent and scaling parameters will be either estimated from observed data if supplied, or set by the user (using keywords in the `bulk_and_scale_parameters` scope)

- custom: If custom is selected, all data names for the xray data will become variable names accessible via the custom interface. The ``custom`` interface allows one to write a small piece of python code that directly works with the python objects them self. Basic knowledge of the cctbx and python are needed to bring this to a fruitful ending. Please contact the authors for detailed help if required. An example is given in the example section.

Examples

Reindexing a data set and model ::

```
xmanip {
  input {
    xray_data {
      file_name = mydata.mtz
      labels = FOBS,SIGFOBS
      write_out = True
    }
    xray_data {
      file_name = mydata.mtz
      labels = R_FREE_FLAG
      write_out = True
    }
    model {
      file_name = mymodel.pdb
    }
  }
  parameters {
    action = reindex
    reindex {
      standard_laws = *niggli
      user_supplied_law = "h,k,l"
    }
  }
  output {
    logfile = "xmanip.log"
    hklout = "reindex.mtz"
    xyzout = "reindex.pdb"
  }
}
```

Applying a symmetry operator to a pdb file ::

```
xmanip {
  input {
    model {
      file_name = mymodel.pdb
    }
  }
  parameters {
    action = manipulate_pdb
    manipulate_pdb {
      task = apply_operator
      apply_operator {
        operator = "x+1/3,y-2/3,z+1/8"
      }
    }
  }
  output {
    logfile = "xmanip.log"
    xyzout = "shifted.pdb"
  }
}
```

Printing out some useful information for an mtz file ::

```

xmanip {
  input {
    xray_data {
      file_name = mydata.mtz
      labels = FOBS,SIGFOBS
      name = fobs
    }
  }
  parameters {
    action = custom
    custom{
      code = """
      print &gt;&gt; out, "Printing d_spacings, epsilons and intensities"
      #change amplitude to intensities
      fobs = fobs.f_as_f_sq()
      #get epsilons
      epsilons = fobs.epsilons().data().as_double()
      #get d spacings
      d_hkl = fobs.d_spacings().data()
      #print the lot to a file
      output_file = open("jiffy_result.txt", 'w')
      for ii, eps, dd in zip( fobs.data(), epsilons, d_hkl):
        print &gt;&gt; output_file, ii, eps, dd
      print &gt;&gt; out, "Done"
      """
    }
  }
}

```

List of all available keywords

xmanip input unit_cell = None Unit cell parameters space_group = None space group xray_data Scope defining xray data. Multiple scopes are allowed file_name = None file name labels = None A unique label or unique substring of a label label_appendix = None Label appendix for output mtz filename = None An identifier of this particular miller array write_out = None Determines if this data is written to the output file model A model associated with the miller arrays. Only one model can be defined. file_name = None A model file parameters action = *reindex manipulate_pdb manipulate_miller Defines which action will be carried out. reindex Reindexing parameters. Acts on coordinates and miller arrays. standard_laws = niggli *reference_setting primitive_setting invert user_supplied Choices of reindexing operators. Will be applied on structure and miller arrays. user_supplied_law = 'h,k,l' User supplied operator. manipulate_miller Acts on a single miller array or a set of miller arrays. task = *get_dano get_diso lsq_scale sfcalc custom None Possible tasks output_label_root = None Output label root get_dano Get ||F+| - |F-| from input data. input_data = None get_diso Get |Fder|-|Fnat| native = None Name of native data derivative = None Name of derivative data use_intensities = True Scale on intensities use_weights = True Use experimental sigmas as weights in scaling scale_weight = True Whether or not to scale the sigmas during scaling lsq_scale input_data_1 = None Reference data input_data_2 = None Data to be scaled use_intensities = True Scale on intensities use_weights = True Use experimental sigmas as weights in scaling scale_weight = True Whether or not to scale the sigmas during scaling sfcalc fobs = None Data name of observed data output = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficients use_bulk_and_scale = *as_estimated user_upplied estimate or use parameters given by user bulk_and_scale_parameters Parameters used in the structure factor calculation. Ignored if experimental data is given d_min = 2.0 resolution of the data to be calculated. overall Bulk solvent and scaling parameters k_overall = 0.1 Overall scalar b_cart Anisotropic B values b_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solvent Solvent parameters k_sol = 0.3 Solvent scale b_sol = 56.0 Solvent B custom A custom script that uses miller_array data names as variables. code = None A piece of python code show_instructions = True Some instructions manipulate_pdb Manipulate elements of a pdb file task = set_b apply_operator *None How to manipulate a pdb file set_bb_iso = 30 new B value for all

atomsapply_operatorstandard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix
Possible operatorsuser_supplied_operator = "x,y,z" Actually operator in x,y,z notationinvert = False Invert
operator given above before applying on coordinatesconcatenate_model = False Determines if new chain is
concatenated to old modelchain_id_increment = 1 Cain id
incrementuser_supplied_cartesian_rotation_matrixRotation,translation matrix in cartesian framer = None
Rotational part of operatorrot = None Translational part of operatoroutputOutput fileslogfile = xmanip.log
Logfilehklout = xmanip.mtz Ouptut miller indices and dataxyzout = xmanip.pdb output PDB file

- xmanipinputunit_cell = None Unit cell parametersspace_group = None space groupxray_dataScope defining xray data. Multiple scopes are allowedfile_name = None file namelabels = None A unique label or unique substring of a labellabel_appendix = None Label appendix for output mtz filename = None An identifier of this particular miller arraywrite_out = None Determines if this data is written to the output filemodelA model associated with the miller arrays. Only one model can be defined.file_name = None A model fileparametersaction = *reindex manipulate_pdb manipulate_miller Defines which action will be carried out.reindexReindexing parameters. Acts on coordinates and miller arrays.standard_laws = niggli *reference_setting primitive_setting invert user_supplied Choices of reindexing operators. Will be applied on structure and miller arrays.user_supplied_law = 'h,k,l' User supplied operator.manipulate_millerActs on a single miller array or a set of miller arrays.task = *get_dano get_diso lsq_scale sfcalc custom None Possible tasksooutput_label_root = None Output label rootget_danoGet ||F+| - |F-| from input data.input_data = None get_disoGet |Fder|-|Fnat|native = None Name of native dataderivative = None Name of derivative datause_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scalingslsq_scaleinput_data_1 = None Reference datainput_data_2 = None Data to be scaleduse_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scalingsfcalcfcobs = None Data name of observed dataoutput = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficientsuse_bulk_and_scale = *as_estimated user_applied estimate or use parameters given by userbulk_and_scale_parametersParameters used in the structure factor calculation. Ignored if experimental data is givend_min = 2.0 resolution of the data to be calculated.overallBulk solvent and scaling parametersk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solventSolvent parametersk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent BcustomA custom script that uses miller_array data names as variables.code = None A piece of python codeshow_instructions = True Some instructionsmanipulate_pdbManipulate elements of a pdb filetask = set_b apply_operator *None How to manipulate a pdb filesset_bb_iso = 30 new B value for all atomsapply_operatorstandard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix Possible operatorsuser_supplied_operator = "x,y,z" Actually operator in x,y,z notationinvert = False Invert operator given above before applying on coordinatesconcatenate_model = False Determines if new chain is concatenated to old modelchain_id_increment = 1 Cain id incrementuser_supplied_cartesian_rotation_matrixRotation,translation matrix in cartesian framer = None Rotational part of operatorrot = None Translational part of operatoroutputOutput fileslogfile = xmanip.log Logfilehklout = xmanip.mtz Ouptut miller indices and dataxyzout = xmanip.pdb output PDB file
- inputunit_cell = None Unit cell parametersspace_group = None space groupxray_dataScope defining xray data. Multiple scopes are allowedfile_name = None file namelabels = None A unique label or unique substring of a labellabel_appendix = None Label appendix for output mtz filename = None An identifier of this particular miller arraywrite_out = None Determines if this data is written to the output filemodelA model associated with the miller arrays. Only one model can be defined.file_name = None A model file
- unit_cell = None Unit cell parameters

- space_group = None space group
- xray_dataScope defining xray data. Multiple scopes are allowedfile_name = None file namelabels = None A unique label or unique substring of a labellabel_appendix = None Label appendix for output mtz filename = None An identifier of this particular miller arraywrite_out = None Determines if this data is written to the output file
- file_name = None file name
- labels = None A unique label or unique substring of a label
- label_appendix = None Label appendix for output mtz file
- name = None An identifier of this particular miller array
- write_out = None Determines if this data is written to the output file
- modelA model associated with the miller arrays. Only one model can be defined.file_name = None A model file
- file_name = None A model file
- parametersaction = *reindex manipulate_pdb manipulate_miller Defines which action will be carried out.reindexReindexing parameters. Acts on coordinates and miller arrays.standard_laws = niggli *reference_setting primitive_setting invert user_supplied Choices of reindexing operators. Will be applied on structure and miller arrays.user_supplied_law = 'h,k,l' User supplied operator.manipulate_millerActs on a single miller array or a set of miller arrays.task = *get_dano get_diso lsq_scale sfcalc custom None Possible taskoutput_label_root = None Output label rootget_danoGet ||F+| - |F-|| from input data.input_data = None get_disoGet |Fder|-|Fnat|native = None Name of native data derivative = None Name of derivative datause_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scalinglsq_scaleinput_data_1 = None Reference datainput_data_2 = None Data to be scaleduse_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scaling sfcalc fobs = None Data name of observed dataoutput = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficientsuse_bulk_and_scale = *as_estimated user_supplied estimate or use parameters given by userbulk_and_scale_parametersParameters used in the structure factor calculation. Ignored if experimental data is givend_min = 2.0 resolution of the data to be calculated.overallBulk solvent and scaling parameterssk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solventSolvent parameterssk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent BcustomA custom script that uses miller_array data names as variables.code = None A piece of python codeshow_instructions = True Some instructionsmanipulate_pdbManipulate elements of a pdb filetask = set_b apply_operator *None How to manipulate a pdb fileset_bb_iso = 30 new B value for all atomsapply_operatorstandard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix Possible operatorsuser_supplied_operator = "x,y,z" Actually operator in x,y,z notationinvert = False Invert operator given above before applying on coordinatesconcatenate_model = False Determines if new chain is concatenated to old modelchain_id_increment = 1 Chain id incrementuser_supplied_cartesian_rotation_matrixRotation,translation matrix in cartesian framer = None Rotational part of operator t = None Translational part of operator
- action = *reindex manipulate_pdb manipulate_miller Defines which action will be carried out.
- reindexReindexing parameters. Acts on coordinates and miller arrays.standard_laws = niggli *reference_setting primitive_setting invert user_supplied Choices of reindexing operators. Will be applied on structure and miller arrays.user_supplied_law = 'h,k,l' User supplied operator.

- standard_laws = niggli *reference_setting primitive_setting invert user_supplied Choices of reindexing operators. Will be applied on structure and miller arrays.
- user_supplied_law = 'h,k,l' User supplied operator.
- manipulate_millerActs on a single miller array or a set of miller arrays.task = *get_dano get_diso lsq_scale sfcalc custom None Possible tasksoutput_label_root = None Output label rootget_danoGet ||F+| - |F-|| from input data.input_data = None get_disoGet |Fder|-|Fnat|native = None Name of native data derivative = None Name of derivative datause_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scalinglsq_scaleinput_data_1 = None Reference datainput_data_2 = None Data to be scaleduse_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scaling sfcalc fobs = None Data name of observed dataoutput = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficientsuse_bulk_and_scale = *as_estimated user_supplied estimate or use parameters given by userbulk_and_scale_parametersParameters used in the structure factor calculation. Ignored if experimental data is givend_min = 2.0 resolution of the data to be calculated.overallBulk solvent and scaling parametersk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solventSolvent parametersk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent BcustomA custom script that uses miller_array data names as variables.code = None A piece of python codeshow_instructions = True Some instructions
- task = *get_dano get_diso lsq_scale sfcalc custom None Possible tasks
- output_label_root = None Output label root
- get_danoGet ||F+| - |F-|| from input data.input_data = None
- input_data = None
- get_disoGet |Fder|-|Fnat|native = None Name of native data derivative = None Name of derivative datause_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scaling
- native = None Name of native data
- derivative = None Name of derivative data
- use_intensities = True Scale on intensities
- use_weights = True Use experimental sigmas as weights in scaling
- scale_weight = True Whether or not to scale the sigmas during scaling
- lsq_scaleinput_data_1 = None Reference datainput_data_2 = None Data to be scaleduse_intensities = True Scale on intensitiesuse_weights = True Use experimental sigmas as weights in scalingscale_weight = True Whether or not to scale the sigmas during scaling
- input_data_1 = None Reference data
- input_data_2 = None Data to be scaled
- use_intensities = True Scale on intensities
- use_weights = True Use experimental sigmas as weights in scaling
- scale_weight = True Whether or not to scale the sigmas during scaling
- sfcalc fobs = None Data name of observed dataoutput = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficientsuse_bulk_and_scale = *as_estimated user_supplied estimate or use parameters given by userbulk_and_scale_parametersParameters used in the structure factor calculation. Ignored if experimental data is givend_min = 2.0 resolution of the data to be calculated.overallBulk

solvent and scaling parametersk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solventSolvent parametersk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent B

- fobs = None Data name of observed data
- output = 2mFo-DFc mFo-DFc *complex_fcalc abs_fcalc intensities Output coefficients
- use_bulk_and_scale = *as_estimated user_applied estimate or use parameters given by user
- bulk_and_scale_parametersParameters used in the structure factor calculation. Ignored if experimental data is givend_min = 2.0 resolution of the data to be calculated.overallBulk solvent and scaling parametersk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0 solventSolvent parametersk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent B
- d_min = 2.0 resolution of the data to be calculated.
- overallBulk solvent and scaling parametersk_overall = 0.1 Overall scalarb_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0
- k_overall = 0.1 Overall scalar
- b_cartAnisotropic B valuesb_11 = 0 b_22 = 0 b_33 = 0 b_12 = 0 b_13 = 0 b_23 = 0
- b_11 = 0
- b_22 = 0
- b_33 = 0
- b_12 = 0
- b_13 = 0
- b_23 = 0
- solventSolvent parametersk_sol = 0.3 Solvent scaleb_sol = 56.0 Solvent B
- k_sol = 0.3 Solvent scale
- b_sol = 56.0 Solvent B
- customA custom script that uses miller_array data names as variables.code = None A piece of python codeshow_instructions = True Some instructions
- code = None A piece of python code
- show_instructions = True Some instructions
- manipulate_pdbManipulate elements of a pdb filetask = set_b apply_operator *None How to manipulate a pdb fileset_bb_iso = 30 new B value for all atomsapply_operatorstandard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix Possible operatorsuser_supplied_operator = "x,y,z" Actually operator in x,y,z notationinvert = False Invert operator given above before applying on coordinatesconcatenate_model = False Determines if new chain is concatenated to old modelchain_id_increment = 1 Cain id incrementuser_supplied_cartesian_rotation_matrixRotation,translation matrix in cartesian framer = None Rotational part of operatorrot = None Translational part of operator
- task = set_b apply_operator *None How to manipulate a pdb file
- set_bb_iso = 30 new B value for all atoms
- b_iso = 30 new B value for all atoms
- apply_operatorstandard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix Possible operatorsuser_supplied_operator = "x,y,z" Actually operator in x,y,z notationinvert = False Invert

operator given above before applying on coordinates
concatenate_model = False Determines if new chain is concatenated to old model
chain_id_increment = 1 Chain id increment
user_supplied_cartesian_rotation_matrix Rotation, translation matrix in cartesian frame = None
Rotational part of operator = None Translational part of operator

- standard_operators = *user_supplied_operator user_supplied_cartesian_rotation_matrix Possible operators
- user_supplied_operator = "x,y,z" Actually operator in x,y,z notation
- invert = False Invert operator given above before applying on coordinates
- concatenate_model = False Determines if new chain is concatenated to old model
- chain_id_increment = 1 Chain id increment
- user_supplied_cartesian_rotation_matrix Rotation, translation matrix in cartesian frame = None
Rotational part of operator = None Translational part of operator
- r = None Rotational part of operator
- t = None Translational part of operator
- output Output files logfile = xmanip.log Logfile hklout = xmanip.mtz Output miller indices and data xyzout = xmanip.pdb output PDB file
- logfile = xmanip.log Logfile
- hklout = xmanip.mtz Output miller indices and data
- xyzout = xmanip.pdb output PDB file